Deep Neural Network

Sangeeta Biswas, Ph.D.

Associate Professor, Dept. of Computer Science & Engineering, University of Rajshahi, Rajshahi, Bangladesh

Artificial Intelligence, Machine Learning, Deep Learning

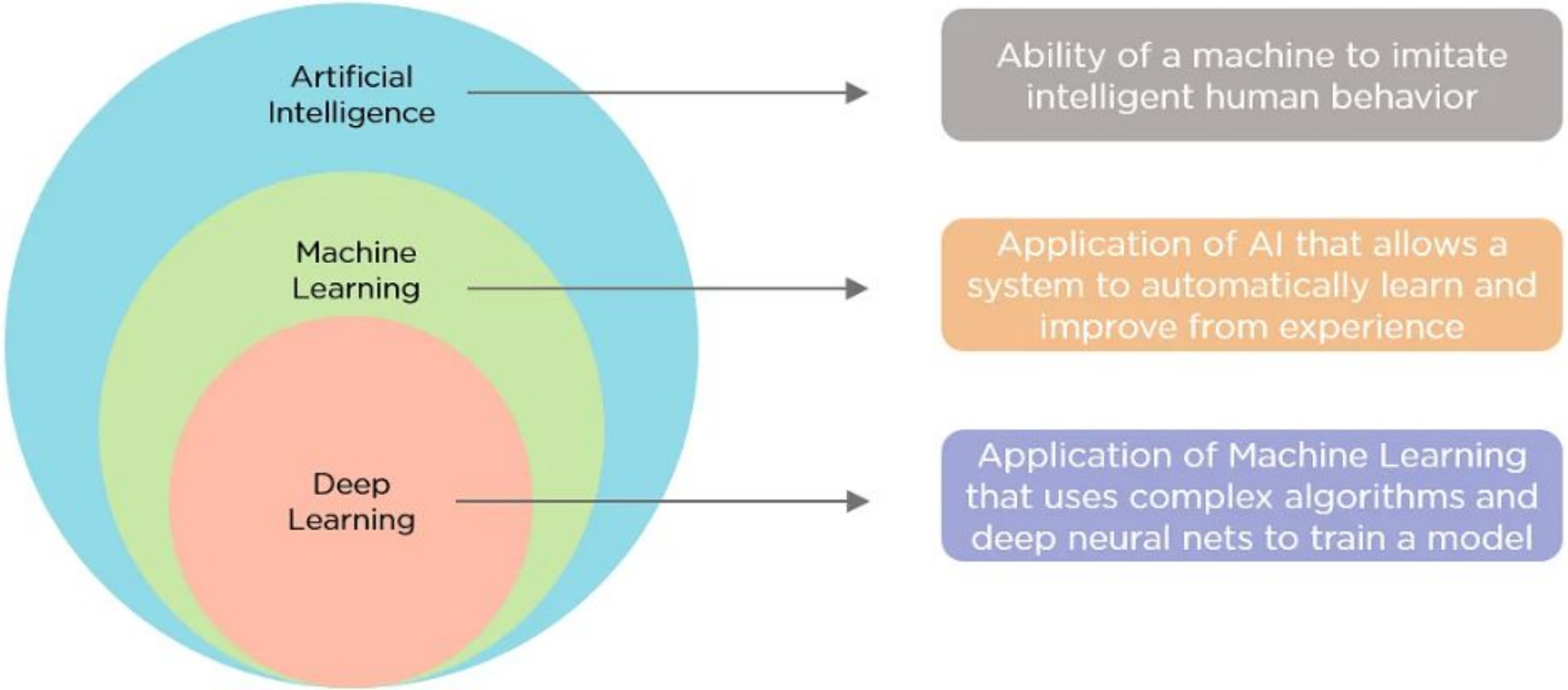
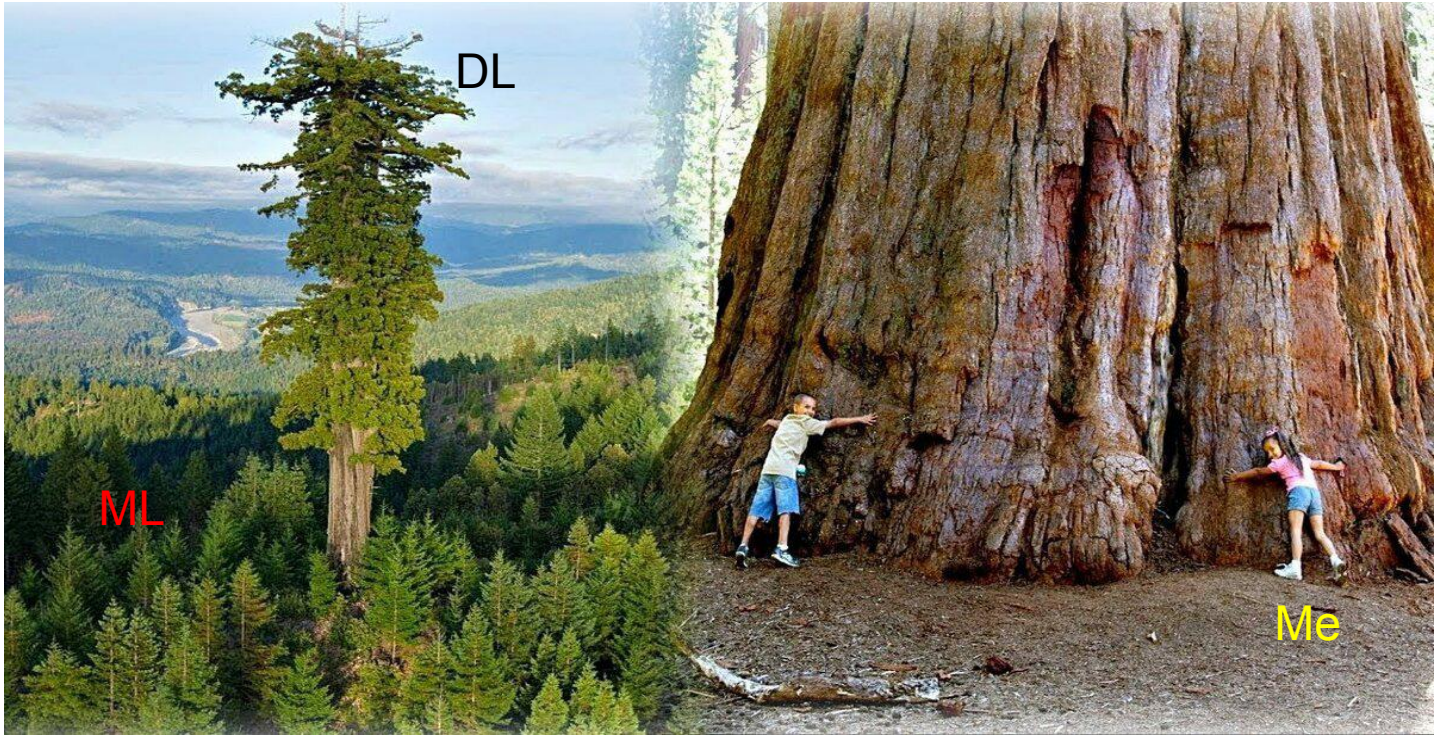


Image Source: Google Search Engine

Machine Learning Vs Deep Learning



Why Deep Learning is So Impressive!

Deep learning achieves:

- Discriminative power
- Predictive Power
- Generative power

can classify or distinguish



Image Source: Google Search Engine

Universal Function Approximator

The main strong point of a deep neural network (DNN) is its universality power.

An DNN can approximate any function no matter how complicated the function is if we can ensure that the network has sufficient number of neurons.

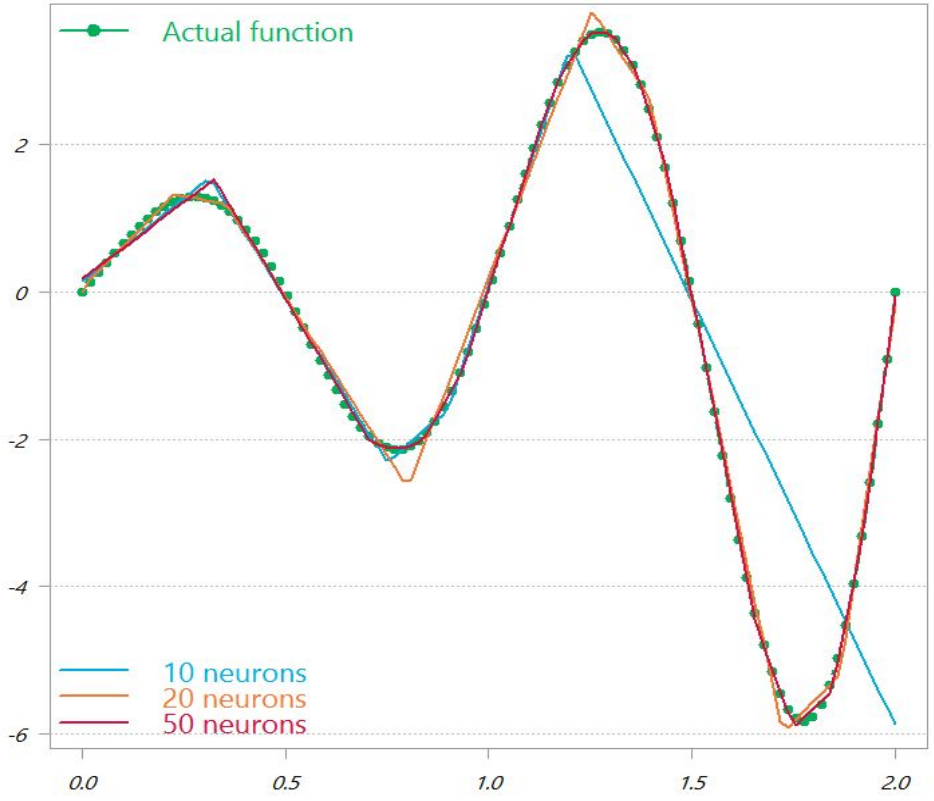


Image Source: Google Search Engine

Artificial Neuron

g is commonly used to mean a function in mathematics.

Artificial neuron is nothing but a composite function, $(g \circ f)(x)$

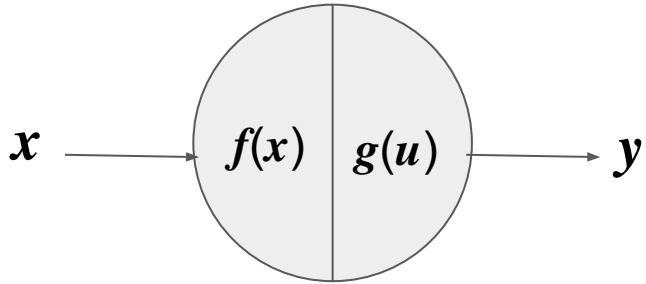
$$u = f(x)$$

$$y = g(u)$$

where

- x is input and y is output
- $f(x)$ is a linear function
- $g(u)$ is generally a nonlinear function

Activation Function



Artificial Neuron

Artificial Neuron

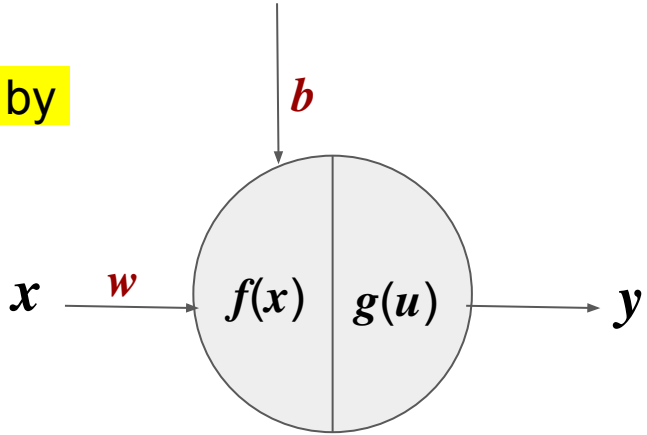
In an artificial neuron, at first input is multiplied by a weight and summed with bias.

$$y = g(f(x))$$

$$u = f(x) = wx + b$$

where

- w is weight which decides how much influence x will have on u.
- b is bias which ensures that u is not too big or too small on average



Artificial Neuron

Artificial Neuron

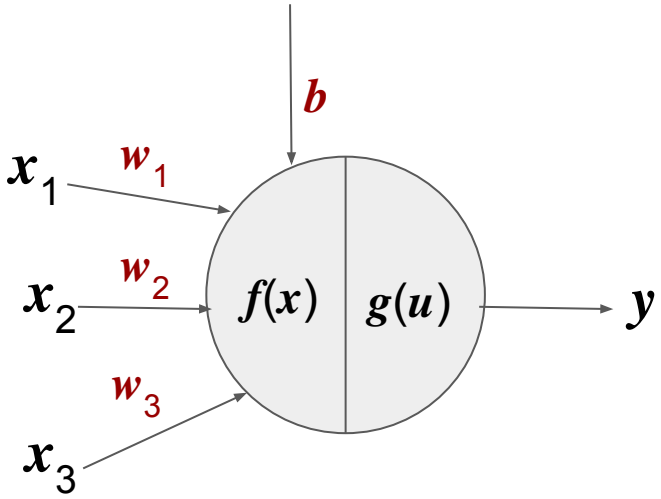
For a vector x, weight w is a vector and bias b is a scalar.

$$u = f(x)$$

$$= w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$= w^T x + b$$

wTx is a dot product, while T itself means transpose.



Artificial Neuron

$g(u)$ is known as **activation function**. Generally nonlinear functions are used as activation functions.

ReLU: g(u)=max(0,u) (If negative, make it zero; if positive, keep it).
Sigmoid: Squashes u to be between 0 and 1.
Tanh: Squashes u to be between -1 and 1.

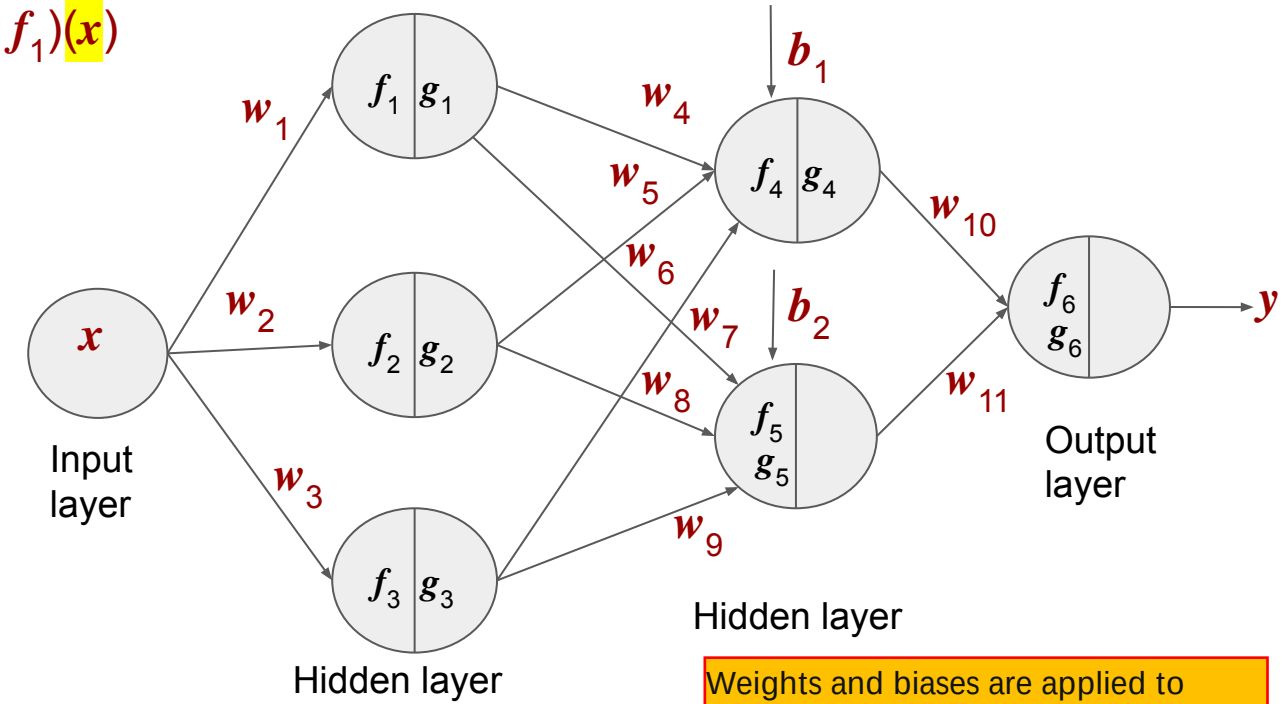
Artificial Neural Network (ANN)

x is only fed to the network, and it can be single or multiplied. Then the value is calculated with linear f and the nonlinear g, and then passed to the next function automatically.

ANN is nothing but a composite of composite functions:

$(g_n \circ f_{n.....} g_2 \circ f_2 \circ g_1 \circ f_1)(x)$

Number of parameters: weights + bias (optional) + any variables in $g(.)$ (optional)



Weights and biases are applied to EVERY layer EXCEPT the input layer.

Parameter Vs Hyperparameter

Parameters: which NN will learn from data

- w, b, any variables in $g(.)$

Hyperparameters: which we need to decide based on our experience, intuition or error-trial such as

- number of layers
- number of neurons in each layer
- activation function
- epoch number
- learning rate

Set manually by you (the engineer) before training begins

Performance Vs Parameters

In general, a model having more parameters performs better

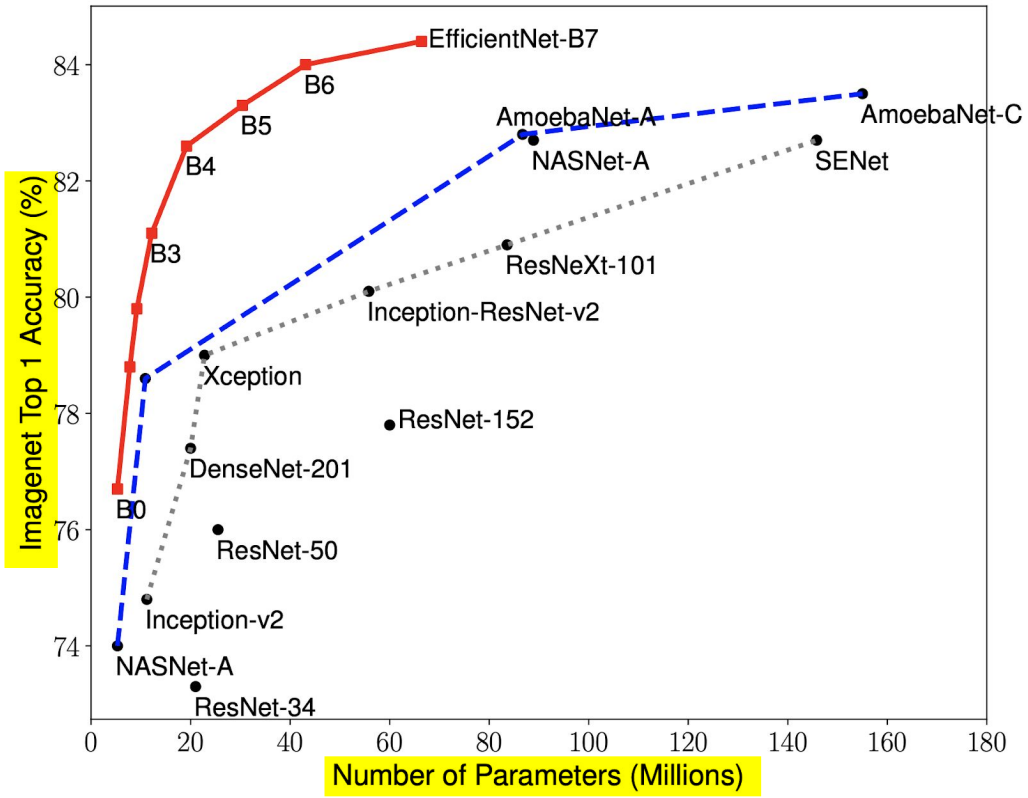


Image Source: Google Search Engine

Training set, Validation Set, Test Set

- **Training Set:** used for finding optimum parameters Model learns from this data (w,b)
- **Validation Set:** used for making decision i.e., finding optimum hyper-parameter Tune layers, neurons, learning rate During training (after each epoch)
- **Test Set:** used for evaluating model performance Only at the VERY END

Be careful about:

- **Over-fitting:** neural network can be easily over-fitted to the training set, therefore, it cannot perform well for unseen data Model memorized the training data
- **Under-fitting:** if we cannot find appropriate hyper-parameter, neural network may not perform well even on the training set

Activation Functions

The activation function takes the raw mathematical result ($u=wx+b$ or $u=wx+b$) and converts it into a signal that is useful for our specific problem.

Depending on the range of output values we need to choose activation function.

LU (Linear Unit) family members such as ReLU, PReLU, ELU, etc., are quite popular as activation functions for the hidden layers.

Depending on what you want the neural network to do, you need the output to be in a specific format. The activation function forces the output into that format.

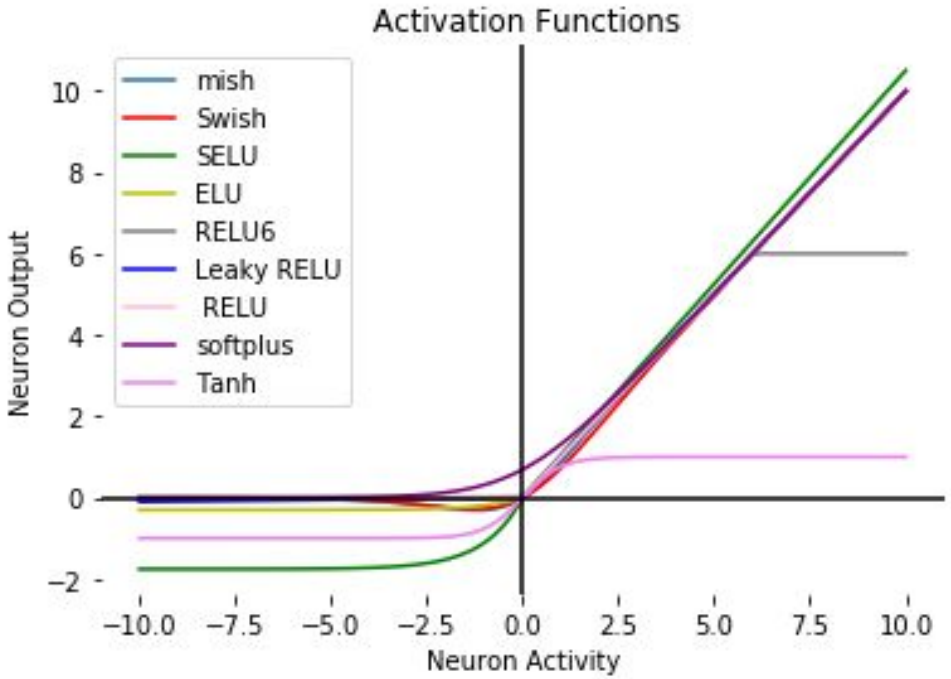


Image Source: Google Search Engine

Sigmoid Vs Softmax

A non-linear converter (like Sigmoid or ReLU) curves, folds, or breaks the line.

The bias gives the activation function something to "grab onto." Without the bias, the neuron is "handcuffed" to the origin (zero).

In output layer, generally:

- Sigmoid is used for binary classification
- Softmax is used for multi-class classification

$$sigmoid,\ y_i = \frac{e^{x_i}}{1 + e^{x_i}}$$

$$softmax,\ y_i = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

Both generate values in the range of 0-1.

Summation of softmax values is 1.

Summation of sigmoid values does not need to be 1.

Loss Functions

A Loss Function (also called Cost Function or Objective Function) is a mathematical formula that calculates the difference between:
The network's predicted output (*y*^)T
he actual correct answer (y)

Different loss functions penalizes wrong predictions differently

Most popular losses: a continuous number

- Regression: Mean-Squared Error (MSE)
- Classification: Log-loss or Cross-Entropy

probability

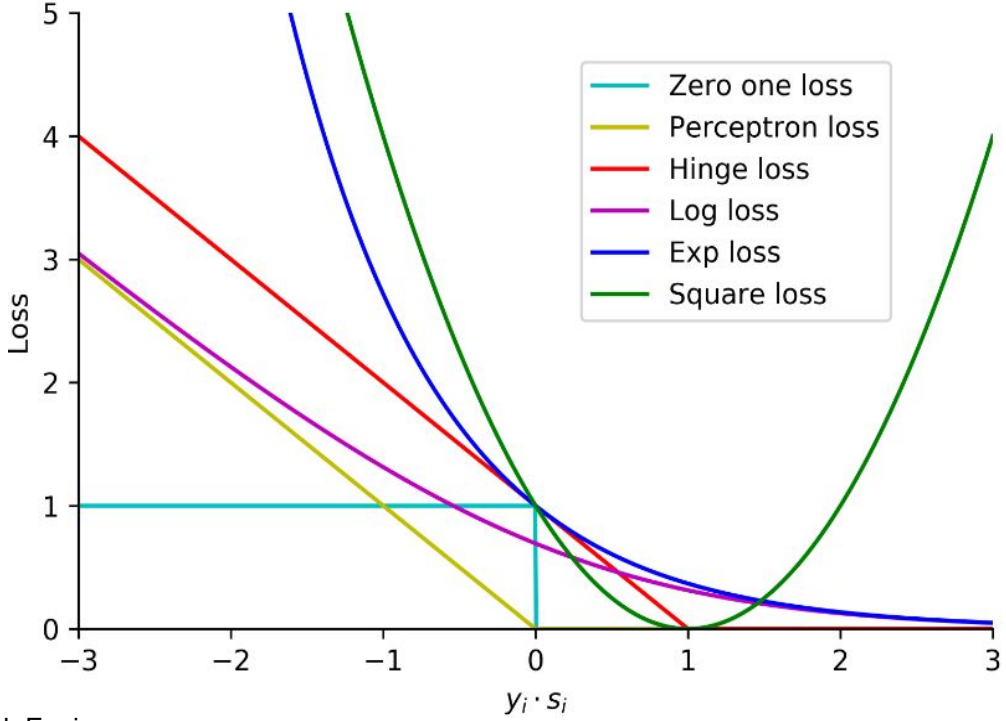
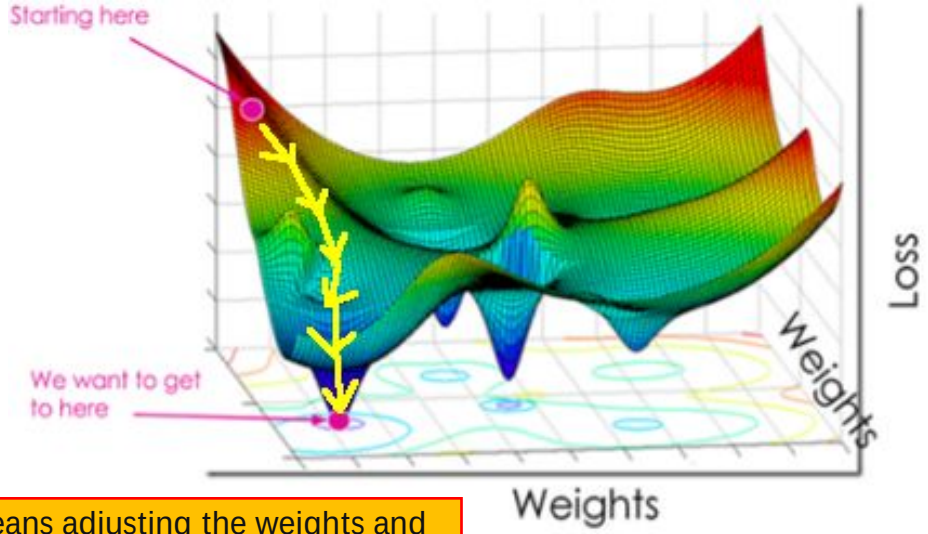


Image Source: Google Search Engine

Training Neural Network

During training we try to find the parameter set for which loss is minimum.
Starting at random values, we update parameters by gradient descent algorithm



Training a neural network means adjusting the weights and biases of all interconnected neurons so that their collective combination transforms any given input (x) into the correct output (y).

Image Source: Google Search Engine

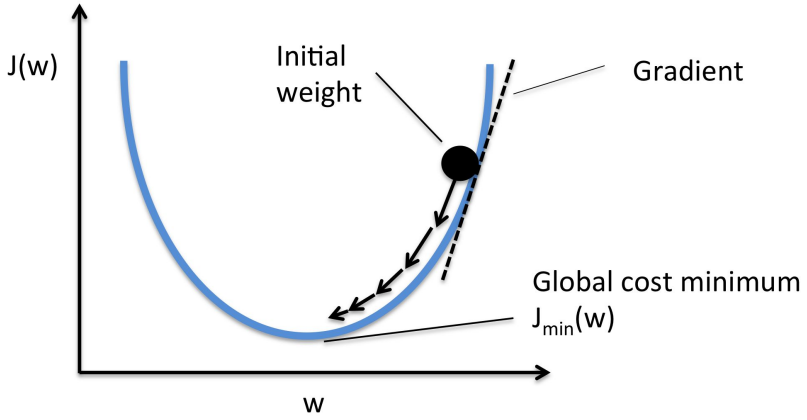
Gradient Descent is the algorithm that finds the best weights and biases by repeatedly taking small steps "downhill" until it reaches the lowest point (minimum loss).

Gradient Descent Algorithm

- Initialize parameters randomly
- Estimate outputs for inputs in a forward pass
- Estimate loss
- Estimate gradient of the loss function with respect to all parameters
- Move parameters in the opposite direction of gradient

$$w = w - \alpha \frac{d}{dw} J(w)$$

- Repeat these steps until loss is minimum



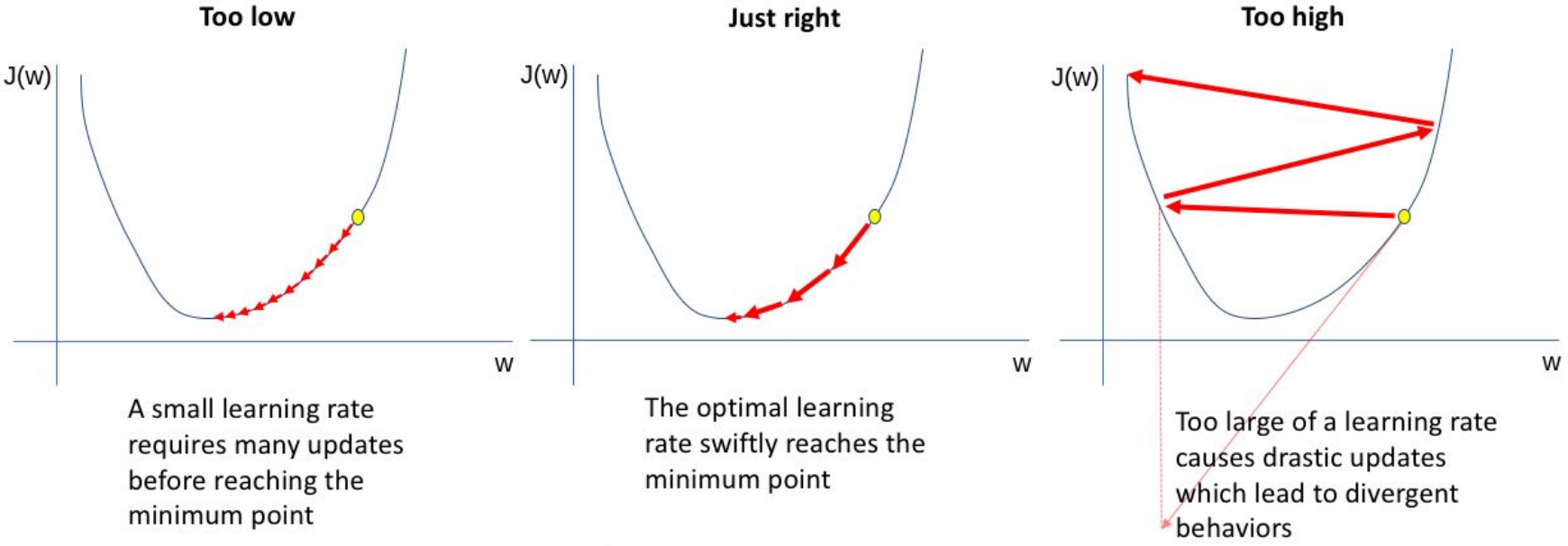
α (Alpha)Learning Rate
$dJ(w)/dw$ Gradient (Slope)
$w - \alpha \times gradient$Update Rule

Gradient = direction + steepness of change
Gradient descent = move opposite to that direction to reduce the loss.

Image Source: Google Search Engine

The learning rate is the most important hyperparameter in deep learning—too small and you'll never get there, too large and you'll fly right past the answer, but just right and you'll reach the minimum quickly and smoothly.

Effect of Learning Rate



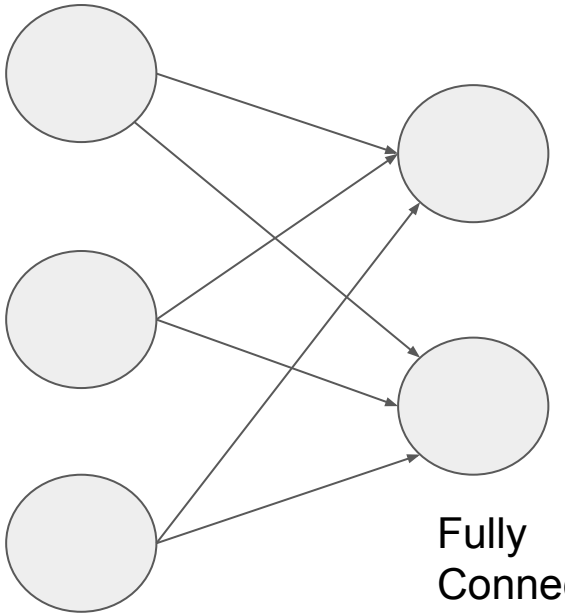
$$w = w - \alpha \frac{d}{dw} J(w)$$

Image Source: Google Search Engine

Fully Connected Layer

A fully connected layer connects every neuron in one layer to every neuron in its previous layer.

An image need to be turned into a vector before feeding into an fully connected layer.



Fully Connected Layer

Fully Connected Neural Network (FCNN)

FCNN consists of a series of fully connected layers

It is also known as multi-layer perceptron (MLP).

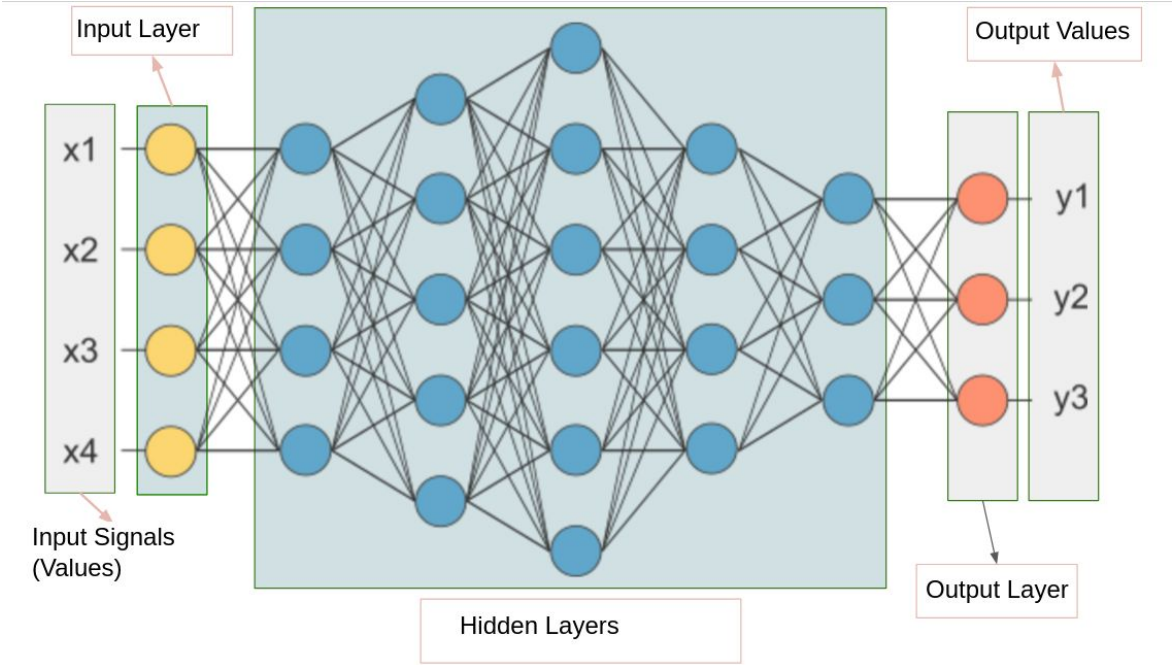
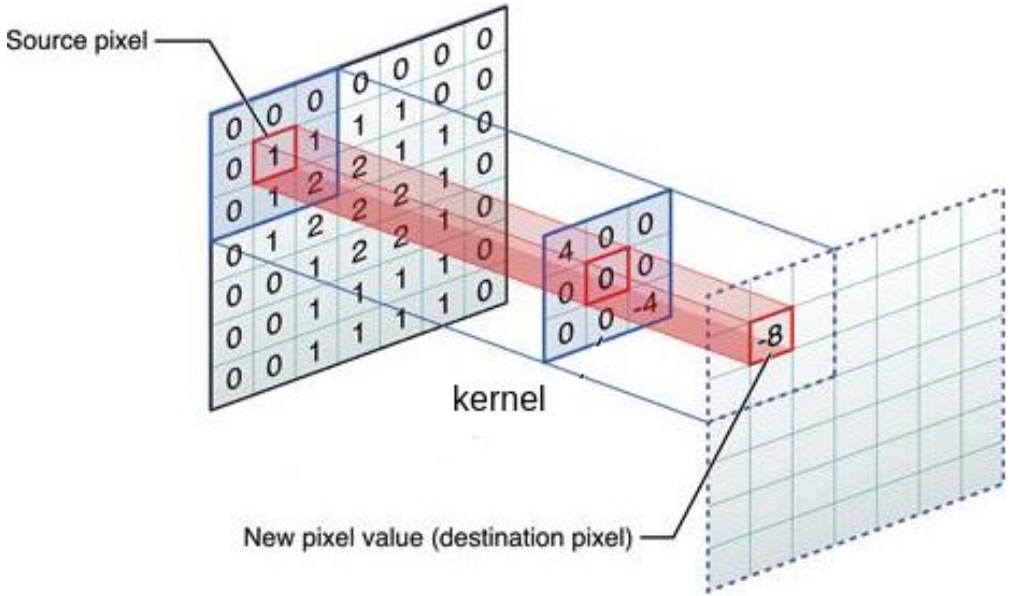


Image Source: Google Search Engine

Conventional Convolution

Convolution layer adds each element of a matrix to its local neighbors, weighted by the kernel.



Source pixel: 0

Destination pixel: 4x0 + 0x0 + 0x0 + 0x0 + 1x0 + 2x0 + 0x0 + 1x0 + (-4)x2 = -8

Total number of multiplication for one pixel = kernel (height x width x channel)

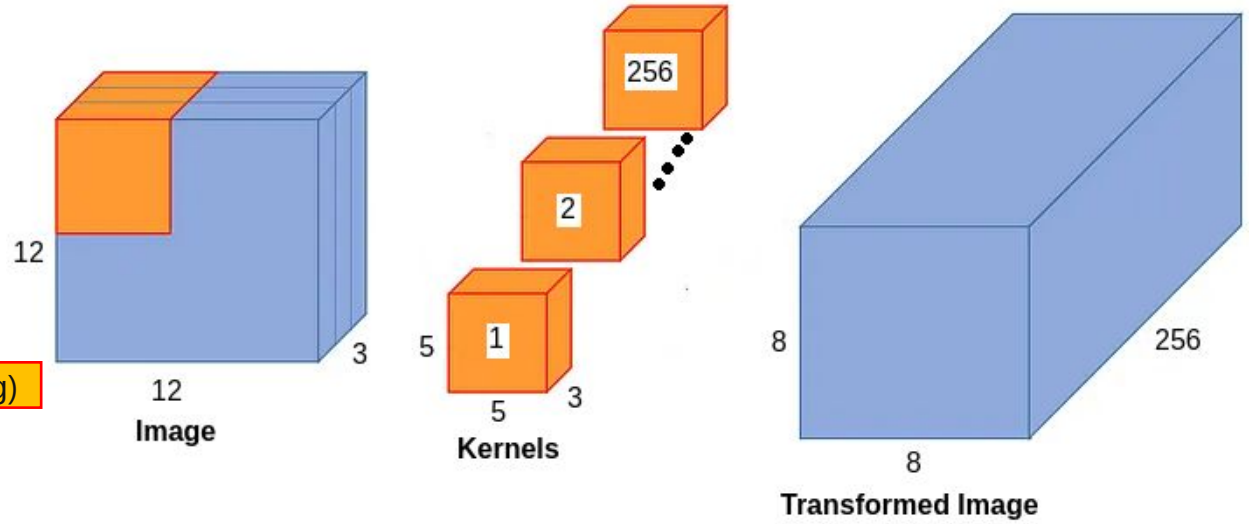
Image Source: Google Search Engine

Convolution Layer

A stencil is a reusable pattern used to create the same shape/design accurately.

Total number of multiplications will depend on:

- kernel size
- number of kernels
- stride (step size while sliding)
- whether zero padding is used or not.



For each pixel & 1 kernel: 5x5x3x1 = 75

For each pixel & 256 kernels: 5x5x3x256 = 19200

For 8x8 pixels & 256 kernels: 8x8x5x5x3x256 = 1228800

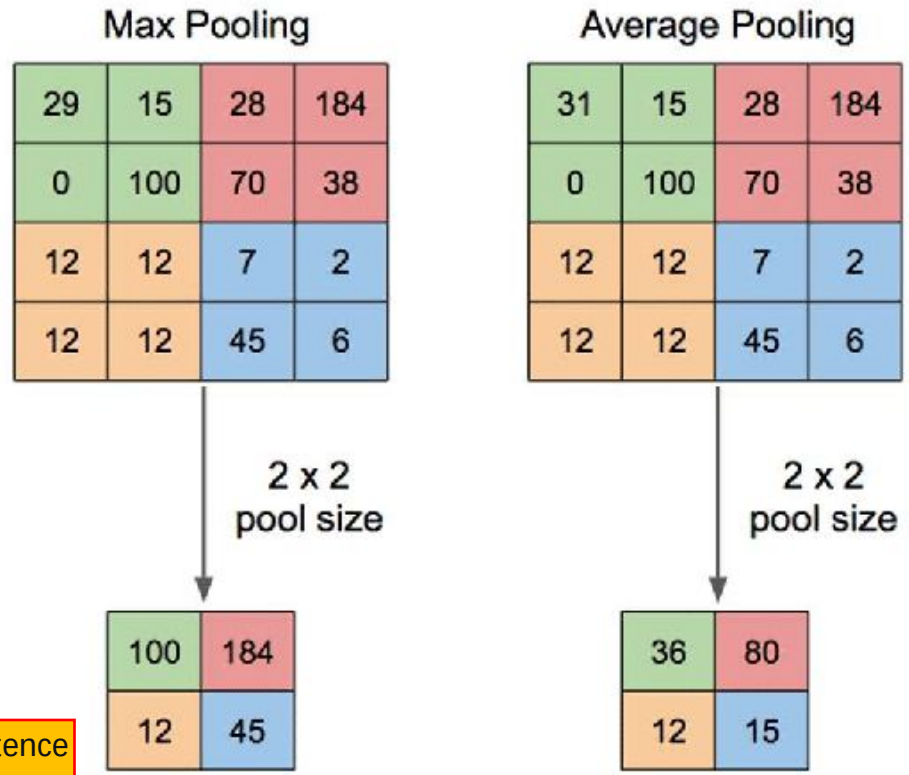
Image Source: Google Search Engine

Pooling shrinks the feature maps by:
Reducing height and width (spatial dimensions)
Keeping the depth (number of channels) unchanged

Pooling Layer

A pooling layer divides the input feature map into a set of non-overlapping regions to:

- reduce the spatial dimensions (i.e., the width and height) of the feature maps
- preserve the depth (i.e., the number of channels)



Pooling is like summarizing a book chapter into a single sentence .You lose the details, but you capture the main idea — and that's exactly what you want for recognizing patterns!

Image Source: Google Search Engine

Convolutional Neural Network (CNN)

CNN mainly consists of a series of convolutional layers. For classification, fully connected layers are added after convolutional layers.

Popular CNNs:

* VGG
* InceptionNet
* ResNet
* EfficientNet
* MobileNet
* XceptionNet
* DenseNet

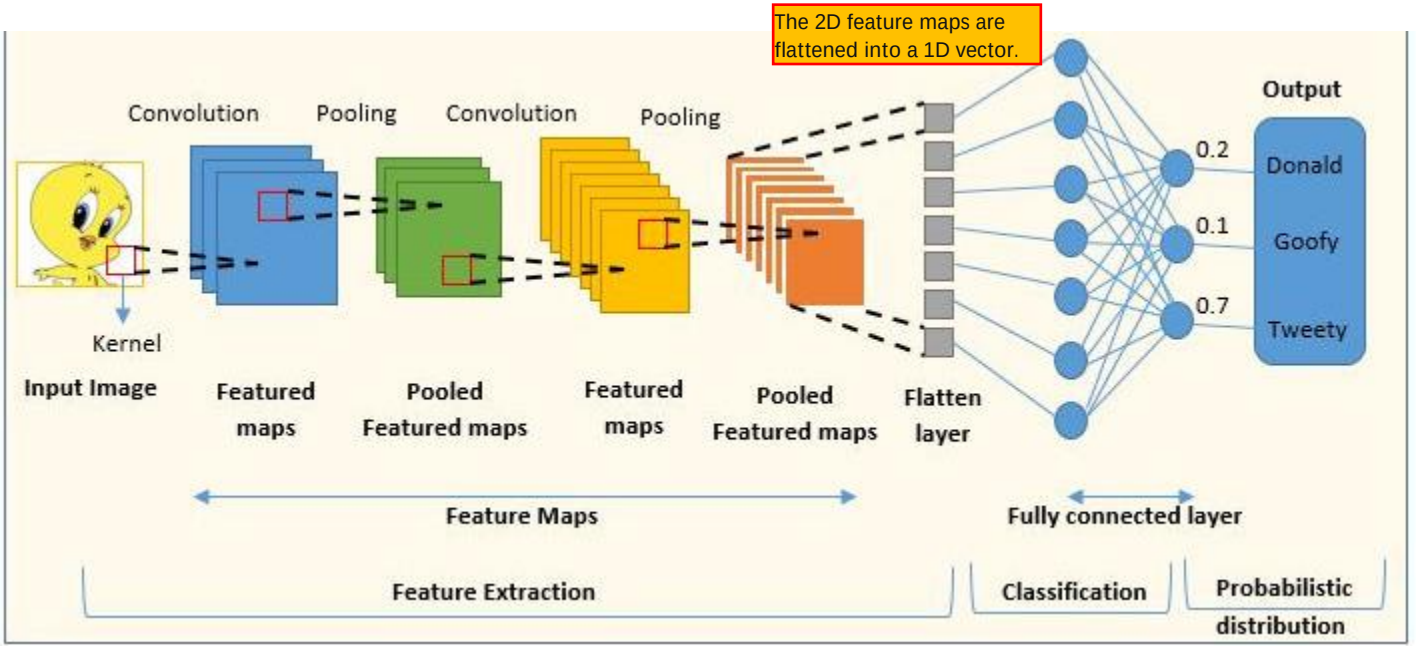


Image Source: Google Search

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3

VGG16

VGG = Visual Geometry Group (at Oxford University)

Key rule: Every time the spatial size halves (2241125628147), the number of channels doubles (64128256512512).

By making it "thick" (512 channels), you give the network 512 different " flashlights" to search for 512 different patterns at the exact same location.

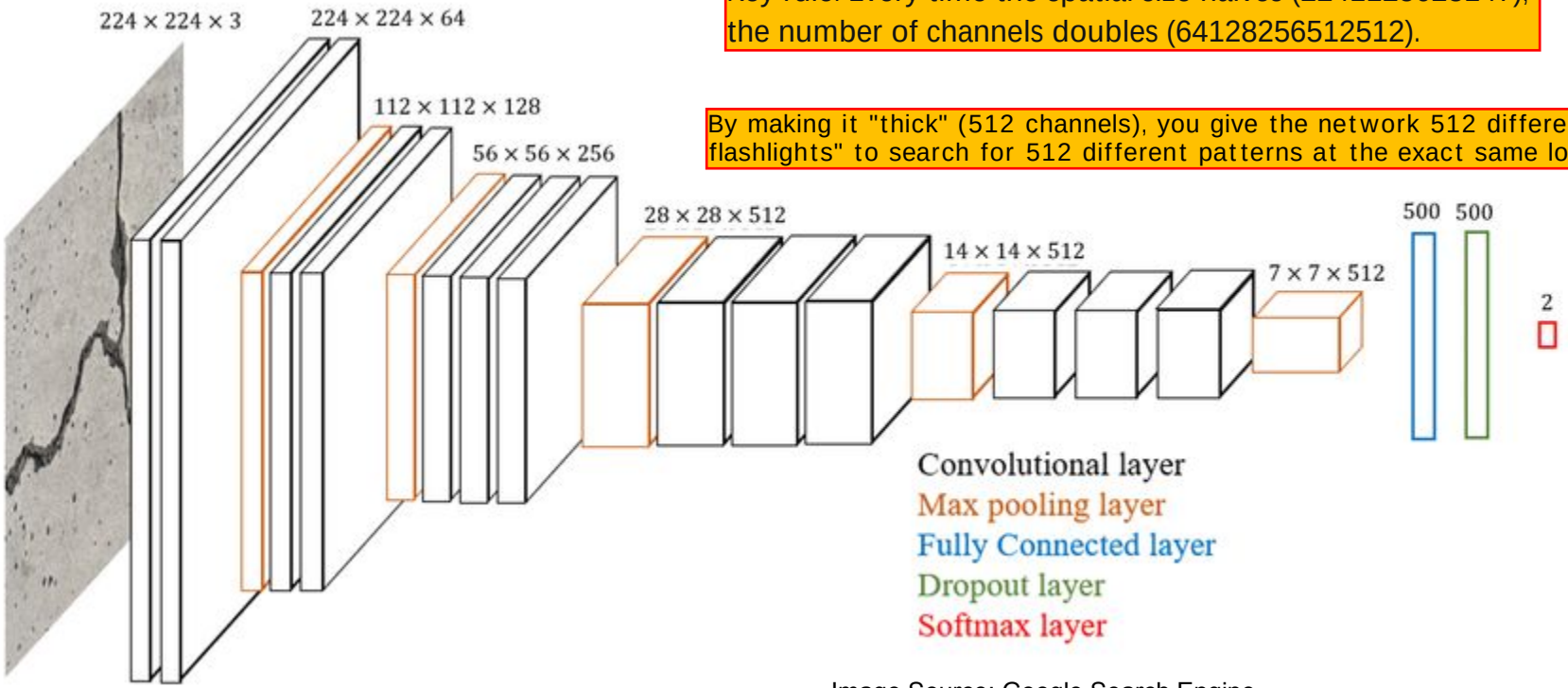
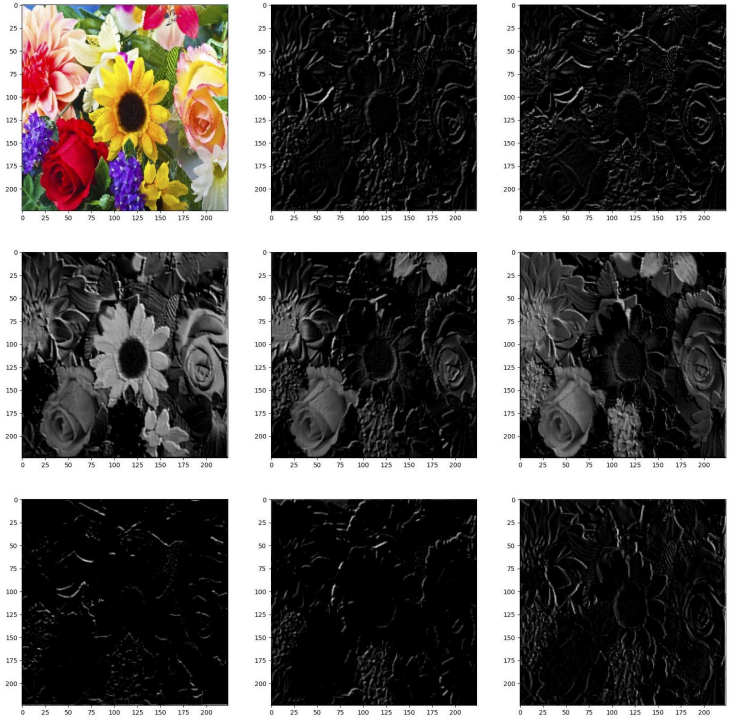


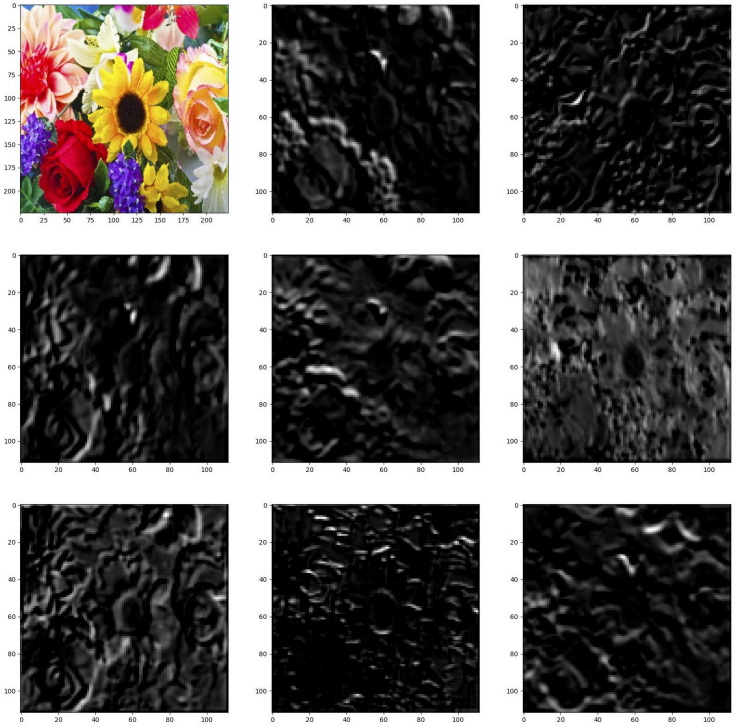
Image Source: Google Search Engine

What VGG16 Sees

More channels = more "memory" and "complexity" to understand highly complex objects



Vgg16_Block1_Conv1



Vgg16_Block2_Conv2

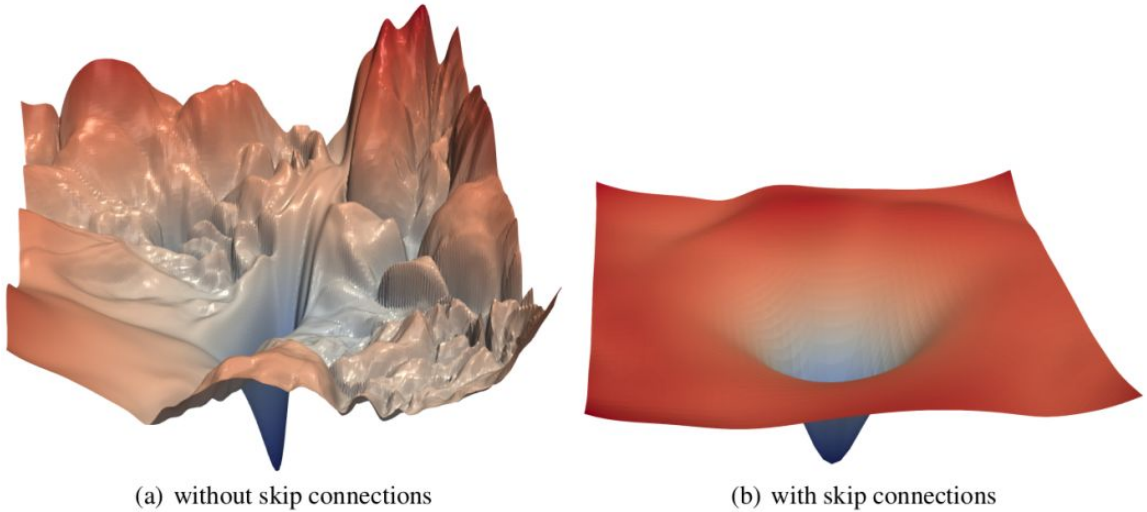
Skip Connections

Instead of forcing data to flow strictly in one direction (layer 1 → layer 2 → layer 3 …), skip connections allow the data to jump over one or more layers and feed directly into a deeper layer.

Skip connections allow CNNs to bypass some layers and connect directly to deeper or shallower ones.

Skip connections lead to:

- faster convergence
- vanishing gradient problem prevention
- loss of information prevention



(a) without skip connections (b) with skip connections

Image Source: Google Search Engine

ResNet

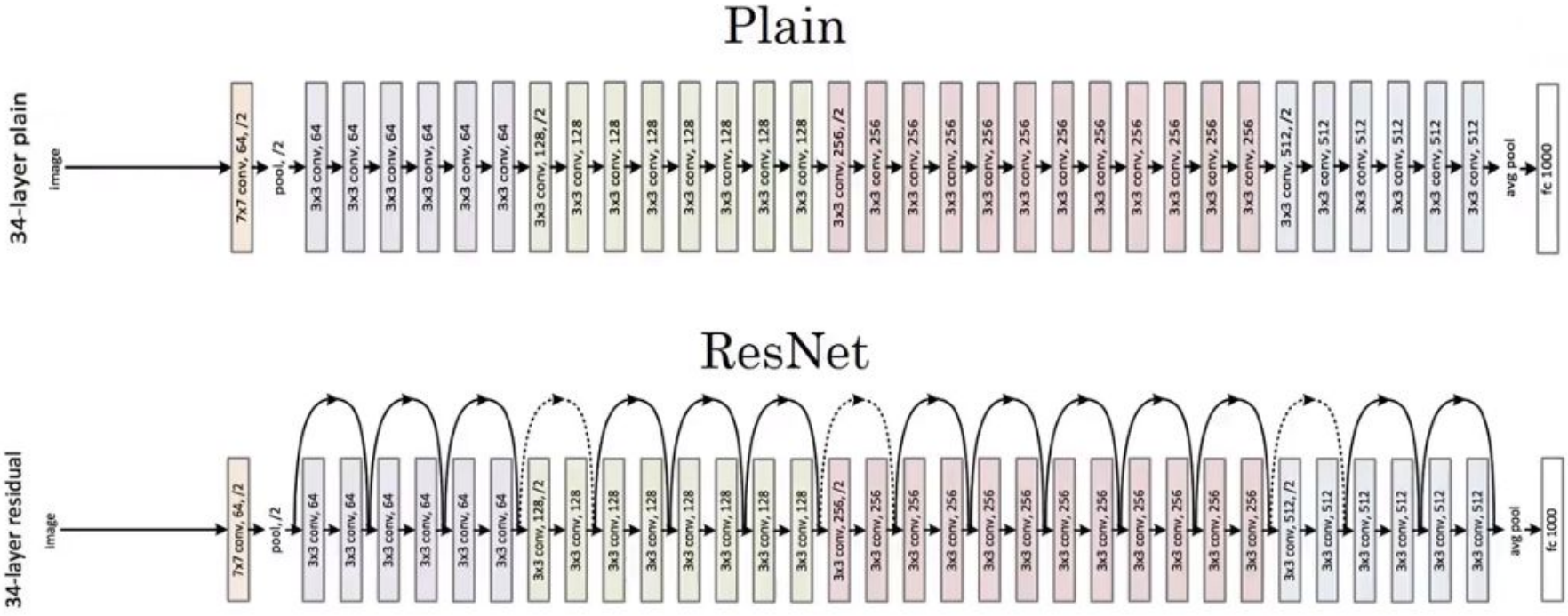


Image Source: Google Search Engine

ResNet Block

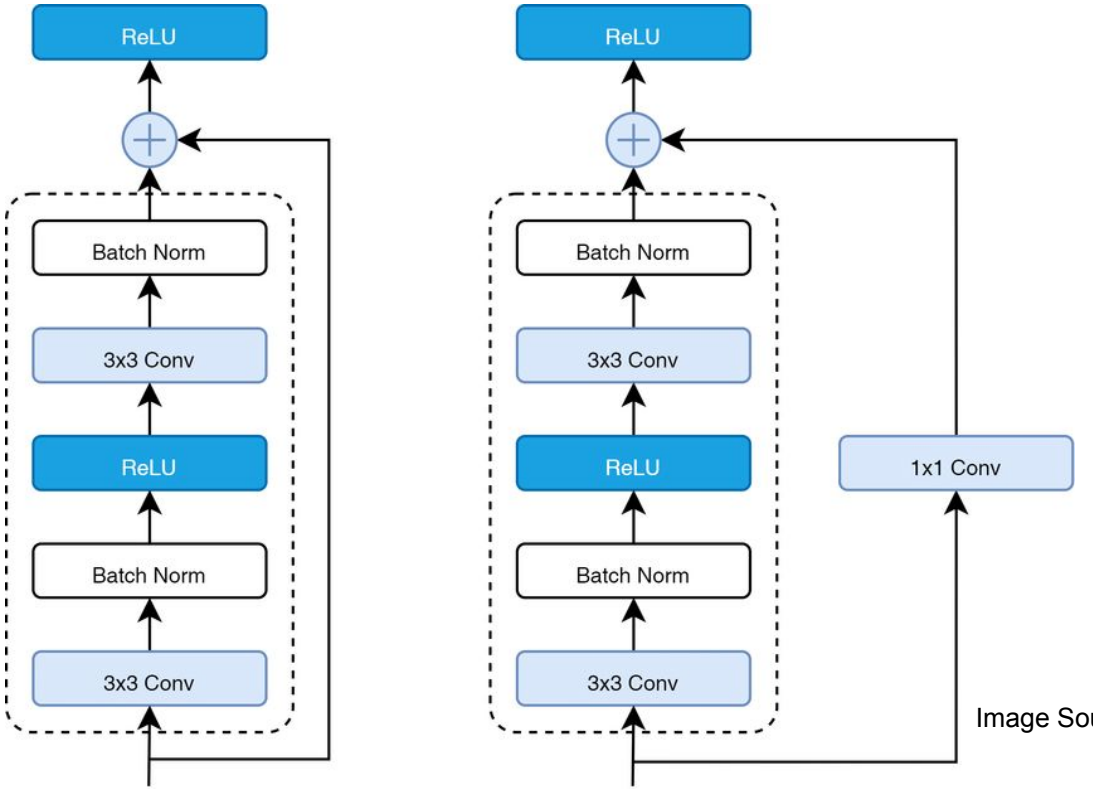
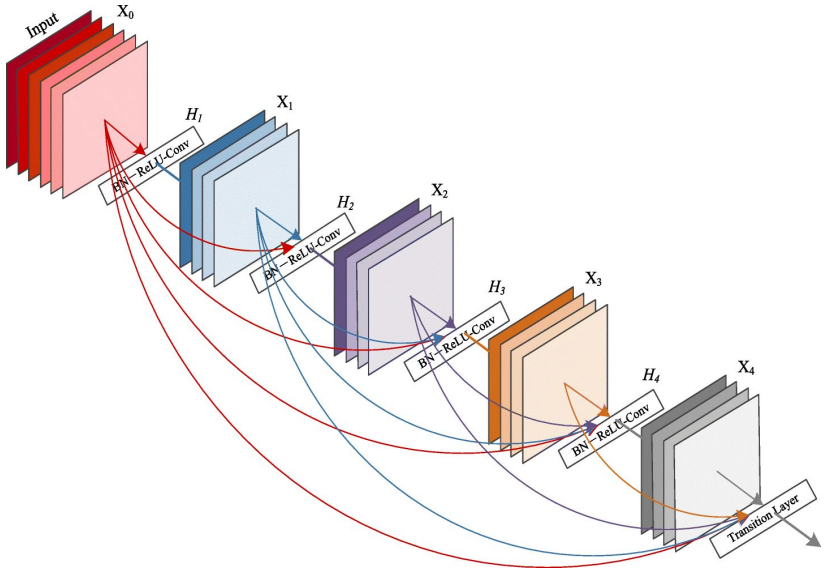


Image Source: Google Search Engine

DenseNet

Skip connections are used for having feature reusability.



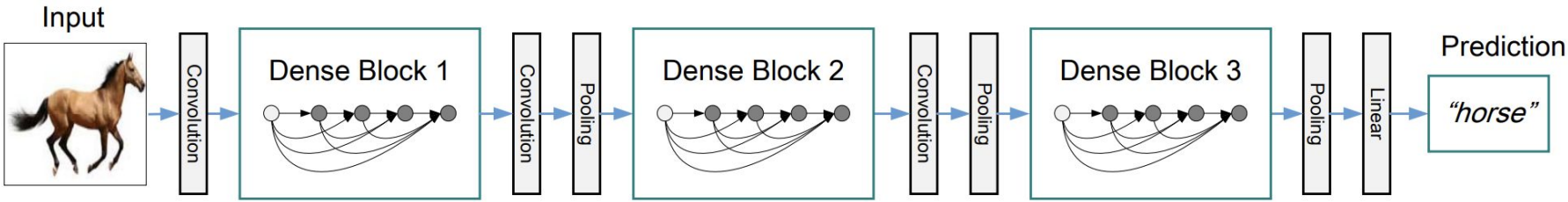


Image Source: Google Search Engine

U-Net

UNet is the most popular network of medical image segmentation.

Skip connections are used for use *fine-grained* details learned in the encoder part to construct an image in the decoder part.

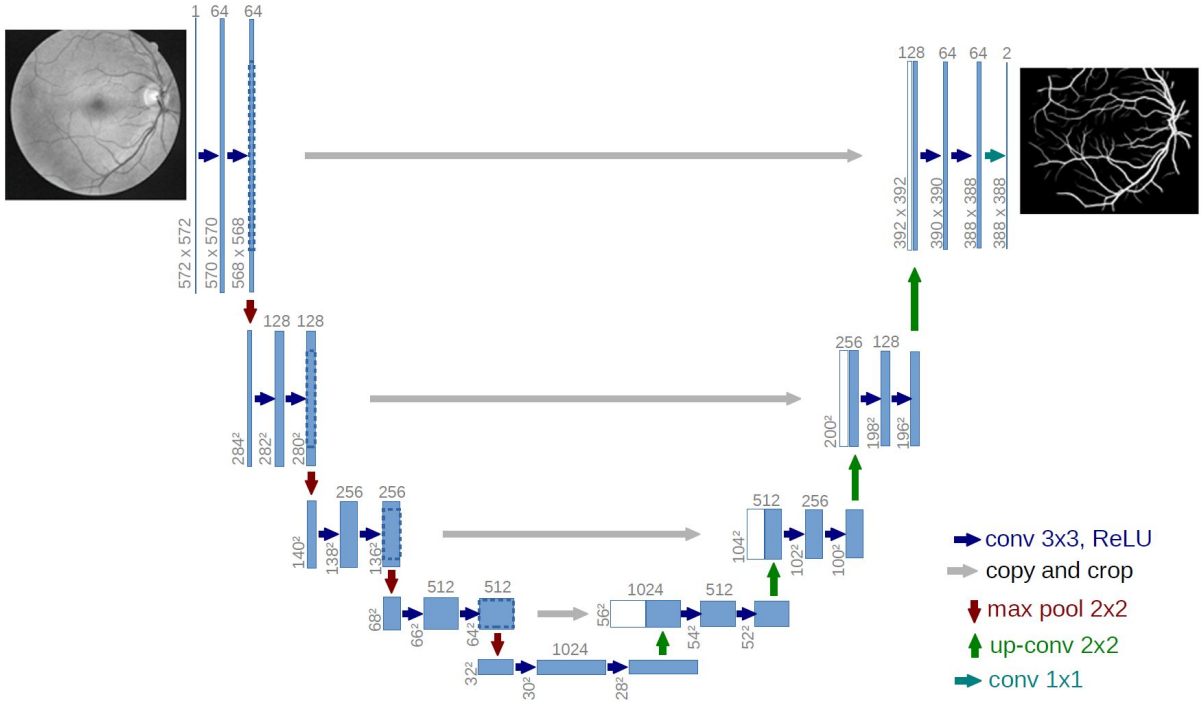


Image Source: Google Search Engine

Point-Wise convolution

1 x 1 kernel is used for point-wise convolution.

Advantage: Increase or reduce the depth (or number of channels) of an image.

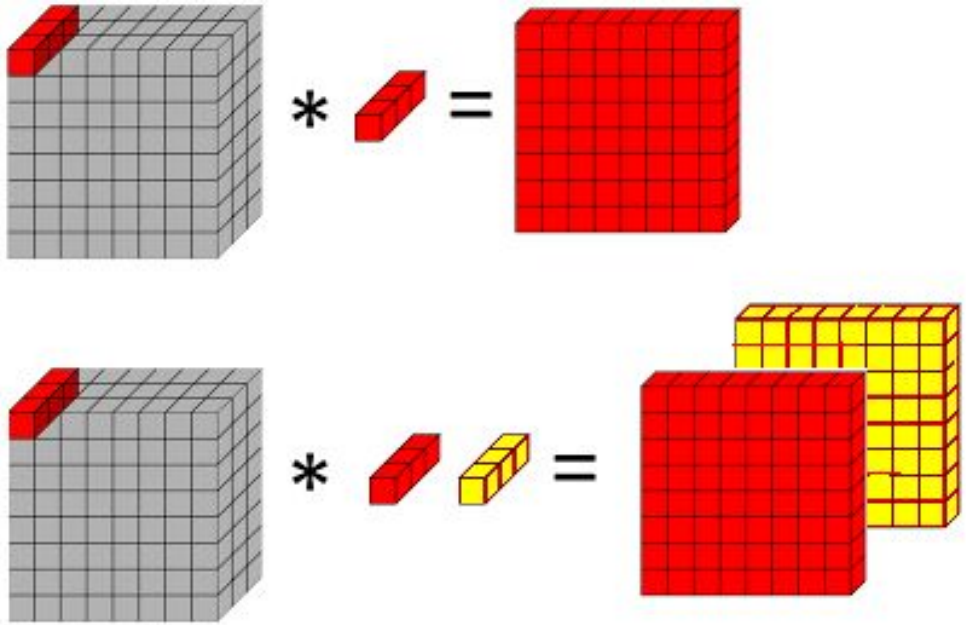


Image Source: Google Search Engine (Modified)

Spatial Separable Convolution

A spatial separable convolution simply divides a kernel into two, smaller kernels.

Advantage: Less parameters, less memory and less computations than regular convolution

Limitations: Not all kernels can be spatially “separated”

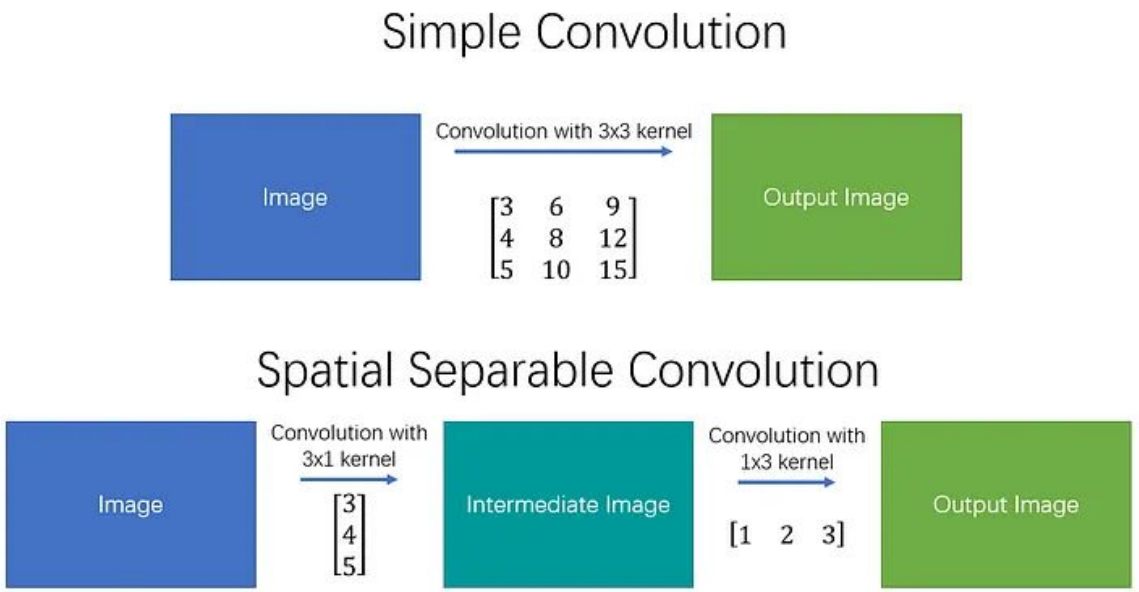


Image Source: Google Search Engine

InceptionNet

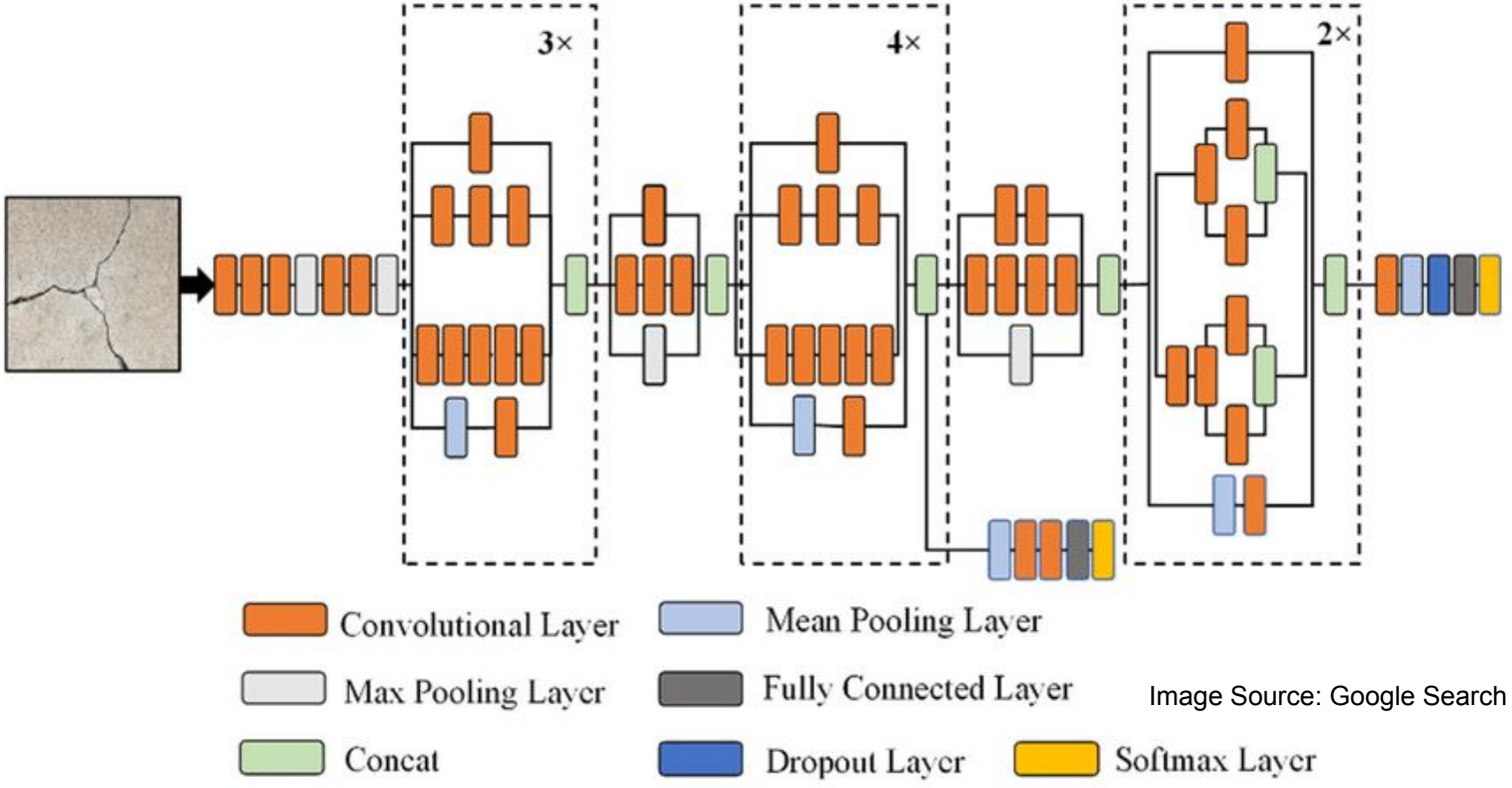


Image Source: Google Search Engine

Inception Modules

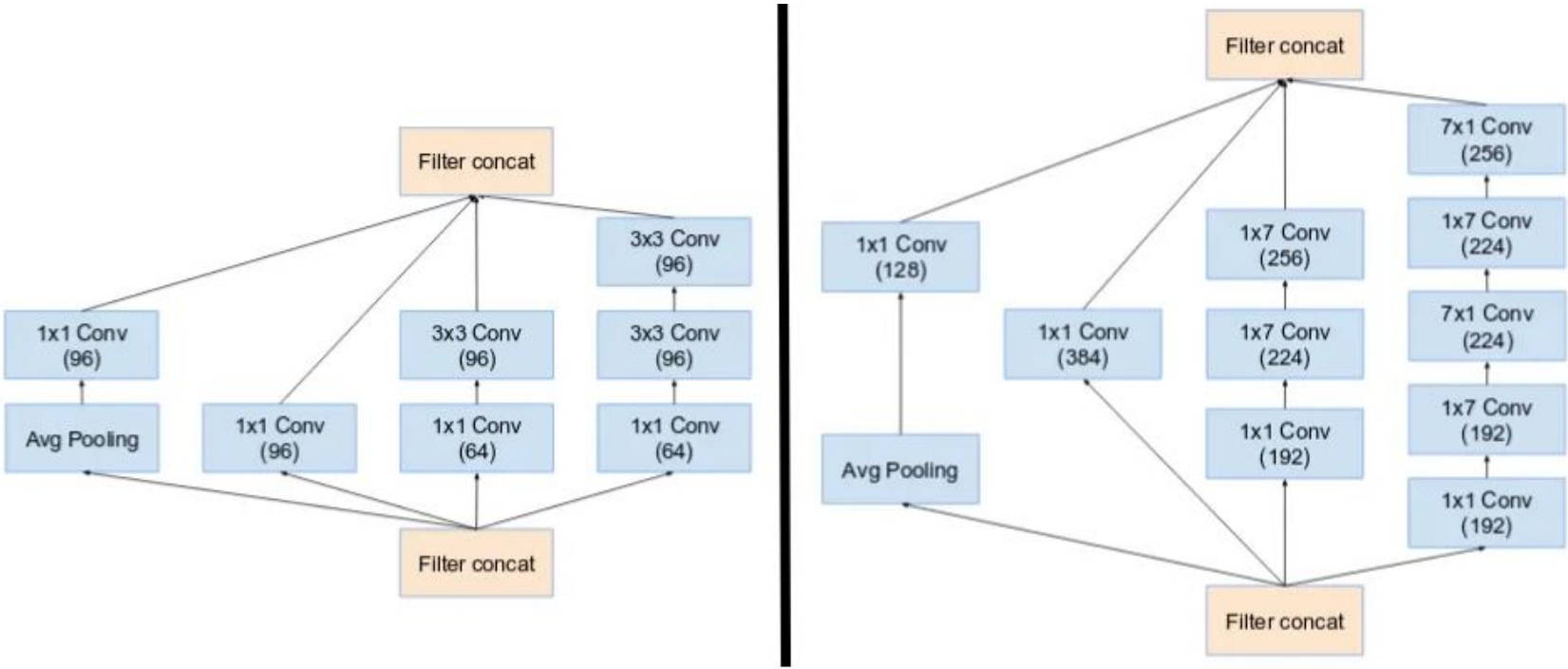


Image Source: Google Search Engine

Depthwise Separable Convolution

Each input channel is convolved with a different kernel (called a depthwise kernel).

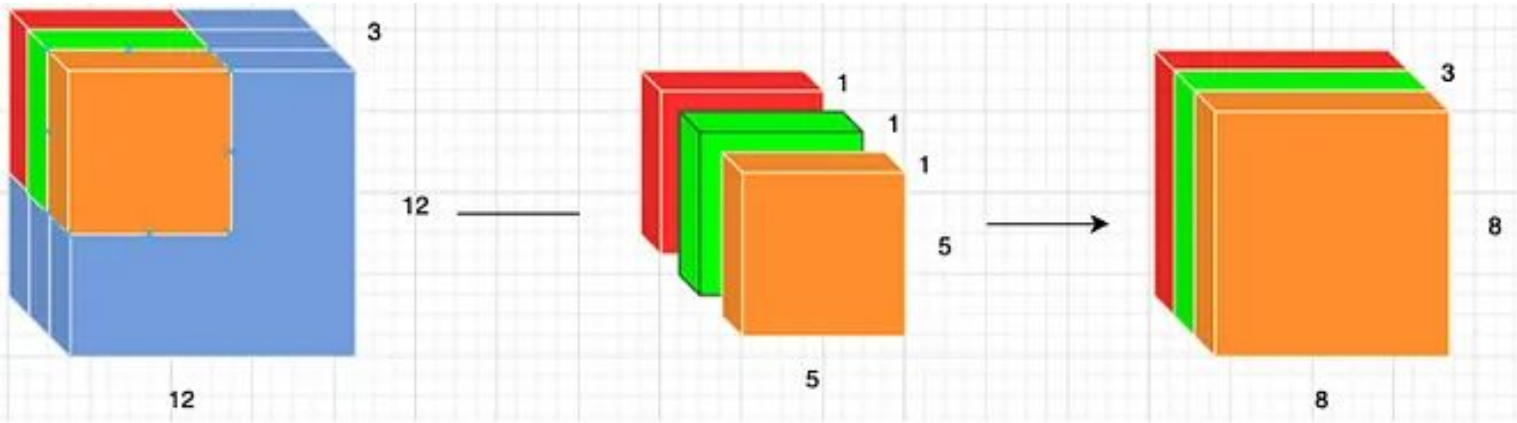


Image Source: Google Search Engine

Depthwise Separable Convolution

Generally, a Depthwise Convolution follows a Pointwise Convolution.

For standard Convolution, number of multiplications: 8x8x5x5x3x256 = 1228800

For Depthwise Separable Convolution, number of multiplications: 8x8x5x5x3 + 256x1x1x3x8x8 = 53952

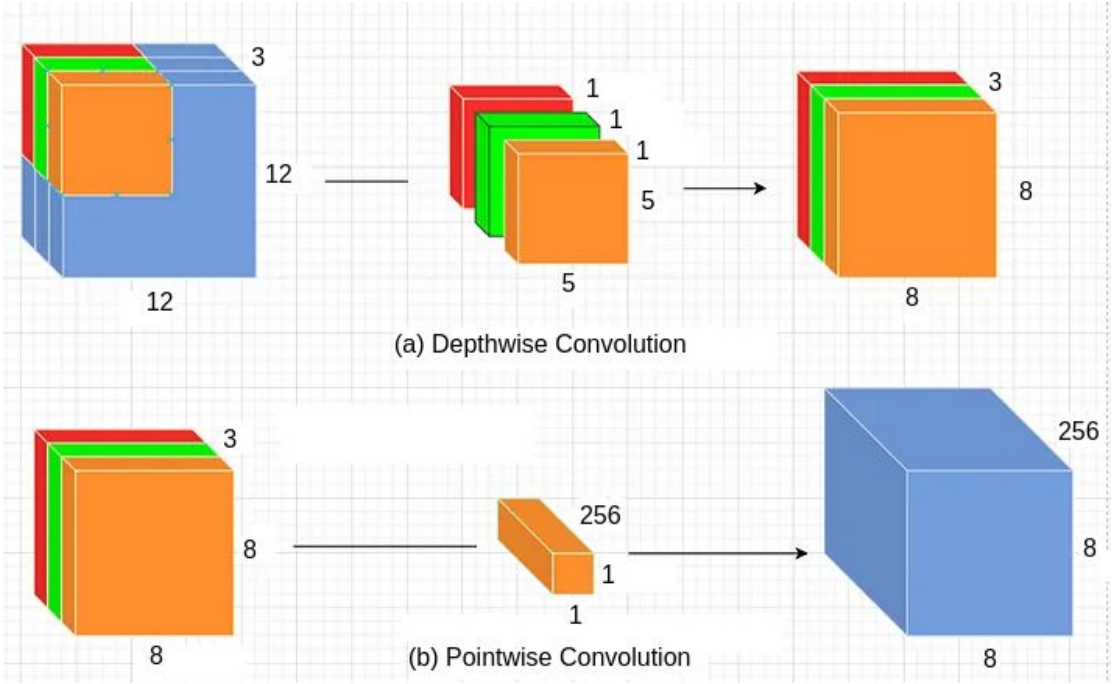


Image Source: Google Search Engine

MobileNet V2

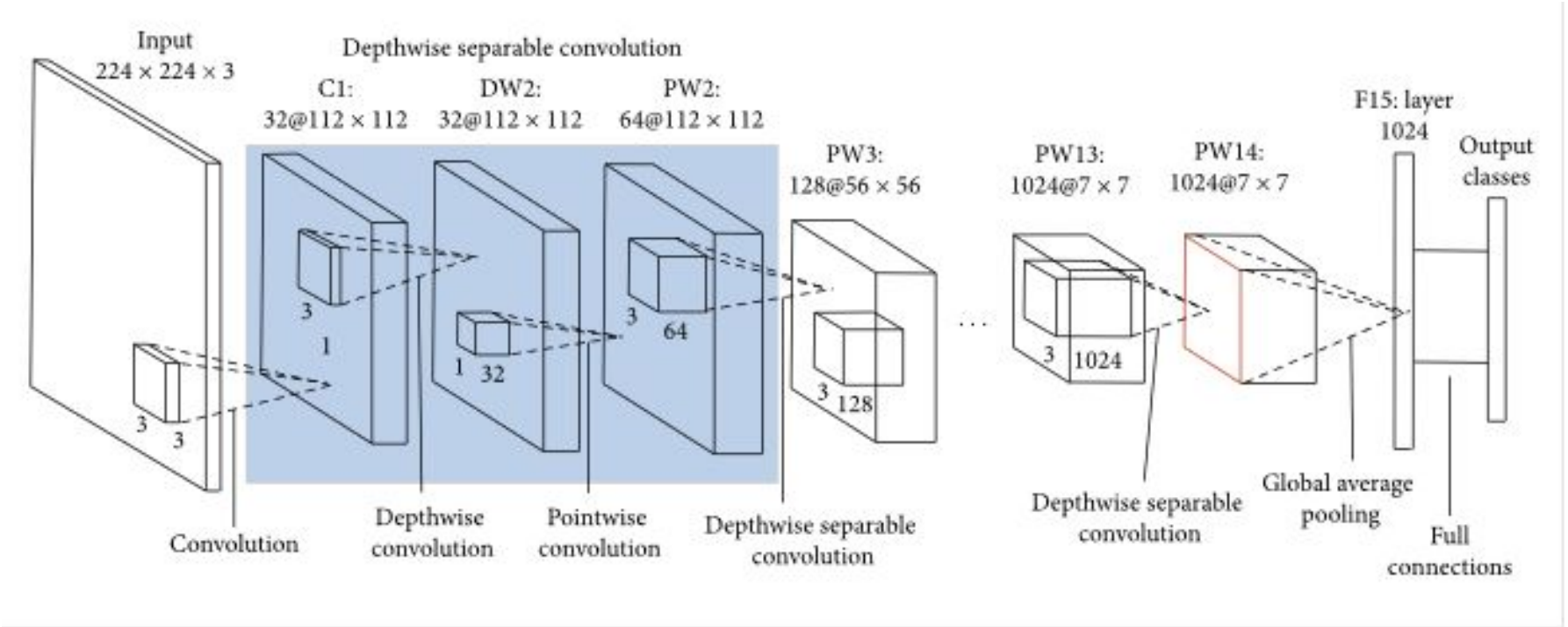


Image Source: Google Search Engine

Modified Depthwise Separable Convolution

In a Modified Depthwise Separable Convolution is the **pointwise convolution followed by a depthwise convolution**.

Xception uses Modified Depthwise Separable Convolution.

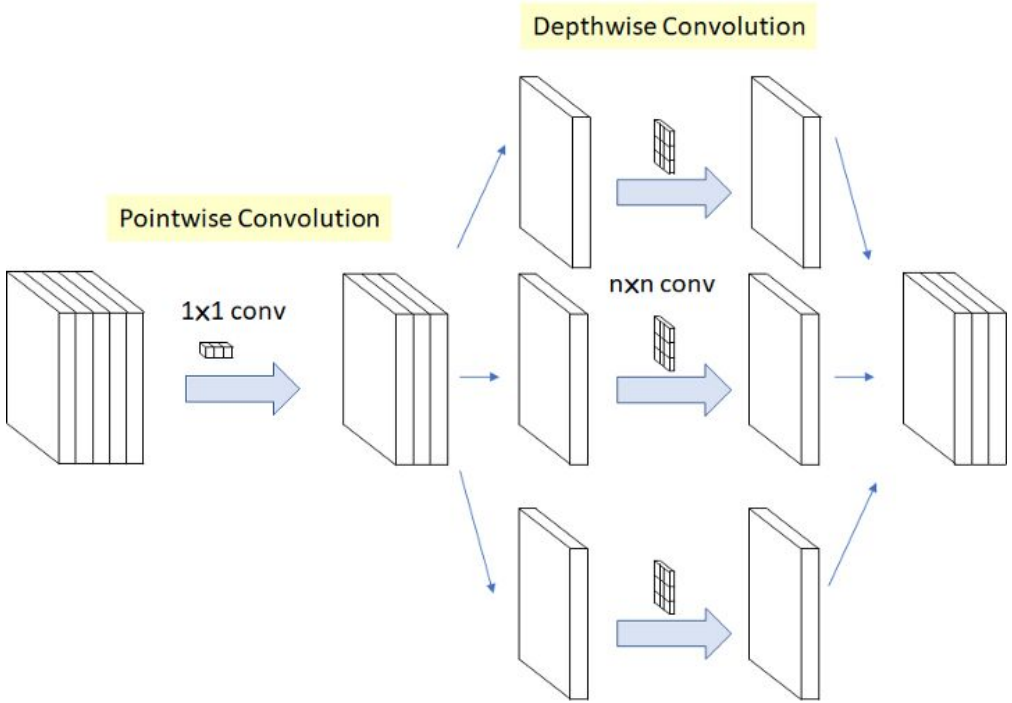


Image Source: Google Search Engine

Xception (Extreme version of Inception)

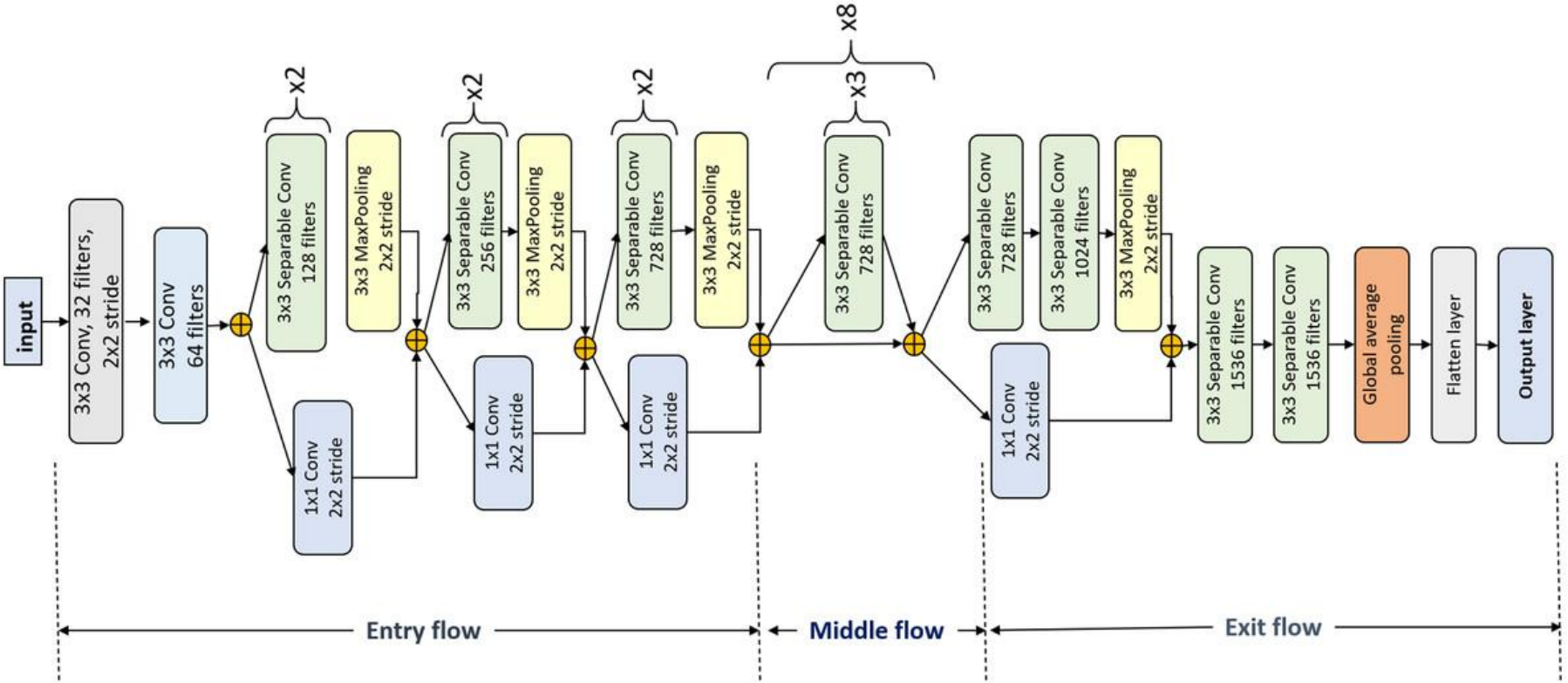


Image Source: Google Search Engine

Transfer Learning (TL)

By TL knowledge learned from a task is re-used in order to boost performance on a related task.

By fine-tuning weights of a pre-trained model are trained on new data.

Generally fine-tuning is started from the conv layers closer to the fully connected layers.

Depending on the amount of data layers need to be fine-tuned.

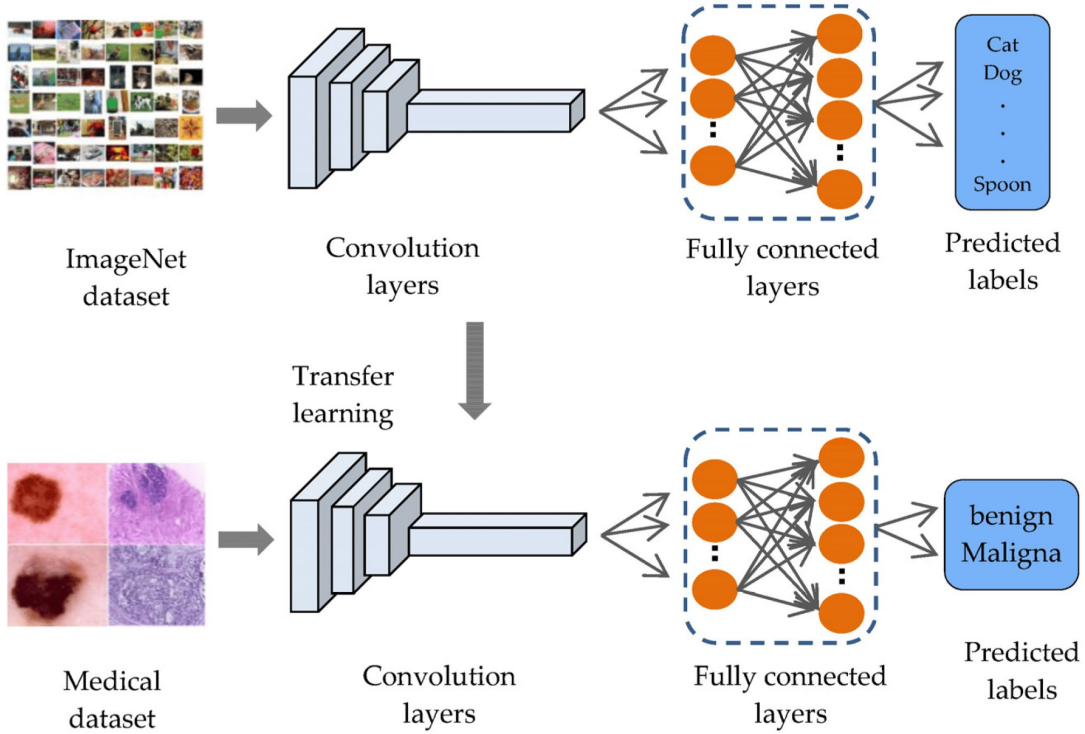


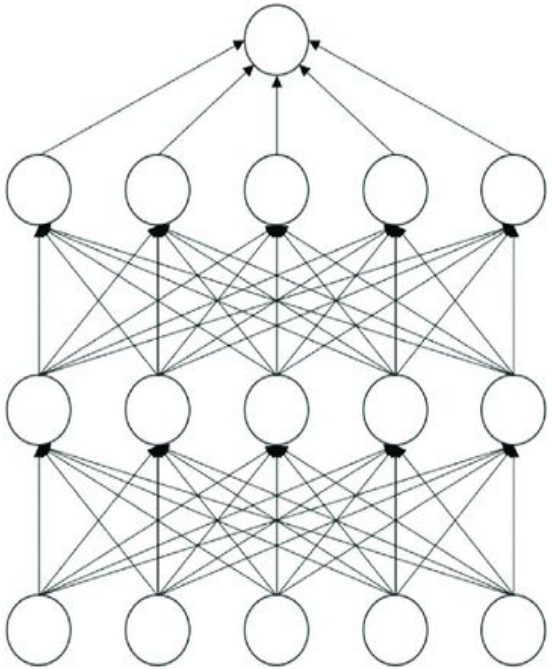
Image Source: Google Search

Randomly "turning off" (dropping out) some neurons during training so the network can't rely on any single neuron.

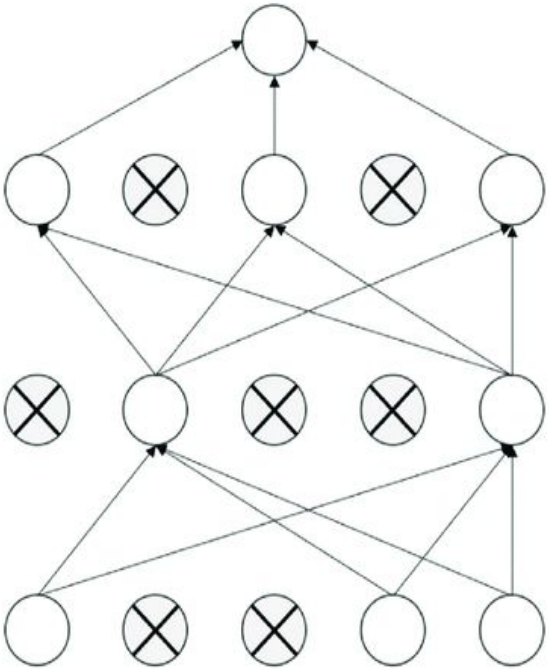
Dropout Layer

In dropout layer:

- individual nodes are excluded in various training runs
- prevents overfitting



Standard Neural Net



After applying dropout

If you always study with the same friend, you might depend on them. Dropout forces you to study alone sometimes, so you actually learn the material yourself!

Image Source: Google Search Engine

Vision Transformer (ViT)

- Transformer first proposed for natural language processing in 2017
- ChatGPT is transformer-based system
- In computer vision, it was successfully implemented in 2020.

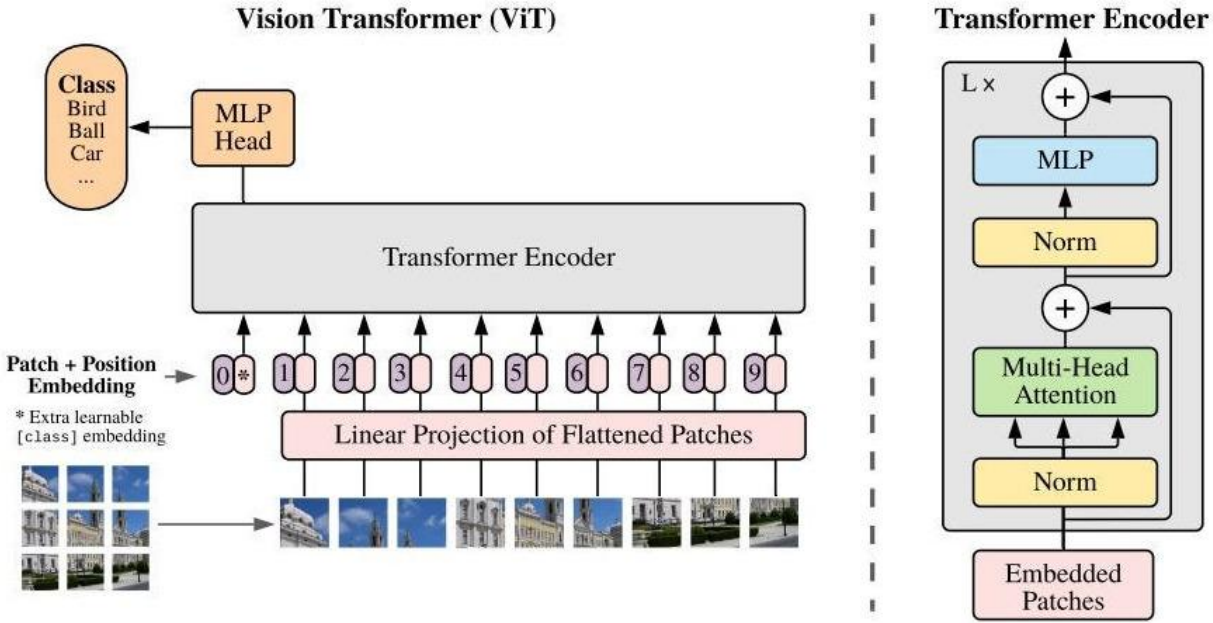


Image Source: Google Search Engine

Transformer is Very Data Hungry Model

- We need huge amount of data to get good performance from ViT
- In small amount of data, it performed worse than CNNs.

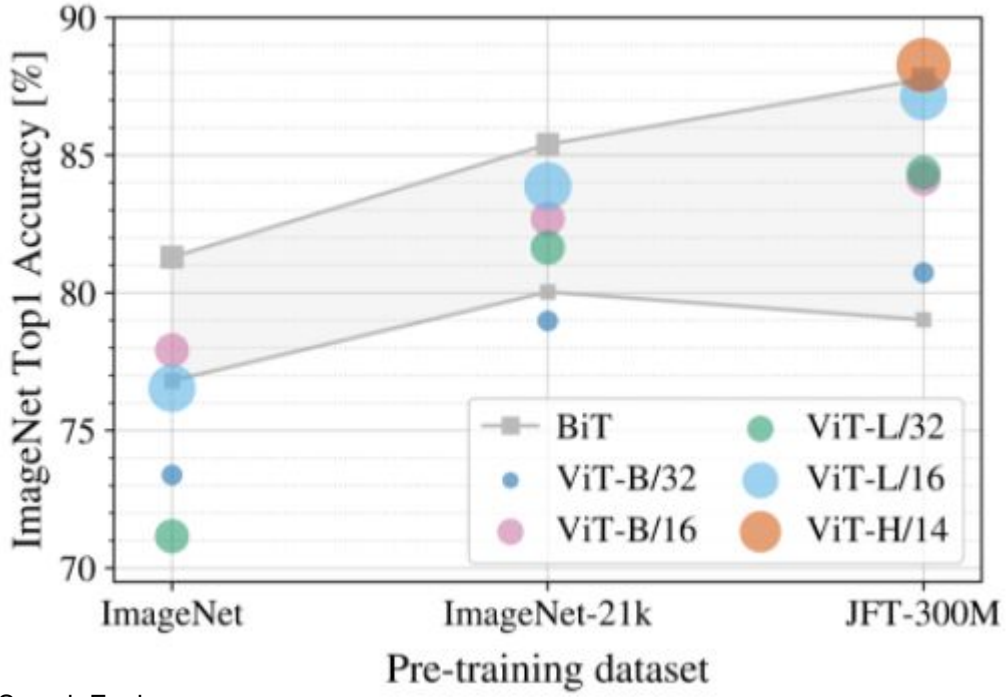


Image Source: Google Search Engine

Types of Training

1. Supervised Training
   - Labeled pairs, i.e., (x_1, y_1), (x_2, y_2), …, (x_n, y_n), are used for training
2. Semi-Supervised Training
   - Little amount of labeled pairs, i.e., (x_1, y_1), (x_2, y_2), …, (x_n, y_n), pairs and a lot of unlabeled data(x_{n+1}, x_{n+2}, x_{n+3},... x_{n+m}) are used for training
3. Self-Supervised Training
   - $((x_1, x_1), 1)$, $((x_1, x_2), 0)$, …, $((x_n, x_n), 1)$, $((x_n, x_1), 0)$, pairs are used for training
4. Unsupervised Training
   - (x_1, x_1), (x_2, x_2), …, (x_n, x_n), pairs can be used for training

Semi-Supervised Training

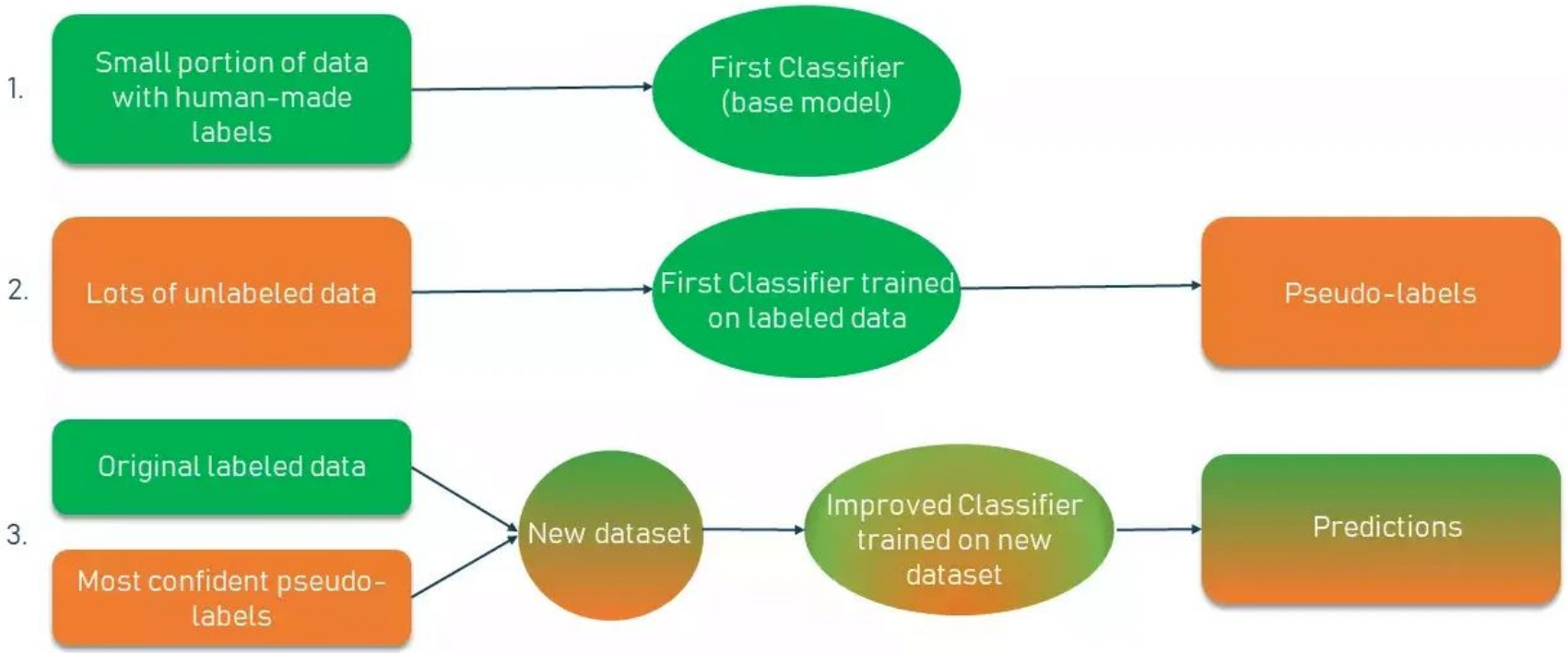


Image Source: Google Search Engine

Encoder-Decoder Architecture

This kind of architecture can be used for:

- Synthesizing images
- Denoising images
- Segmentation
- Unsupervised training

When input images and output images are the same, we call model autoencoder.

Training an autoencoder is an example of unsupervised training.

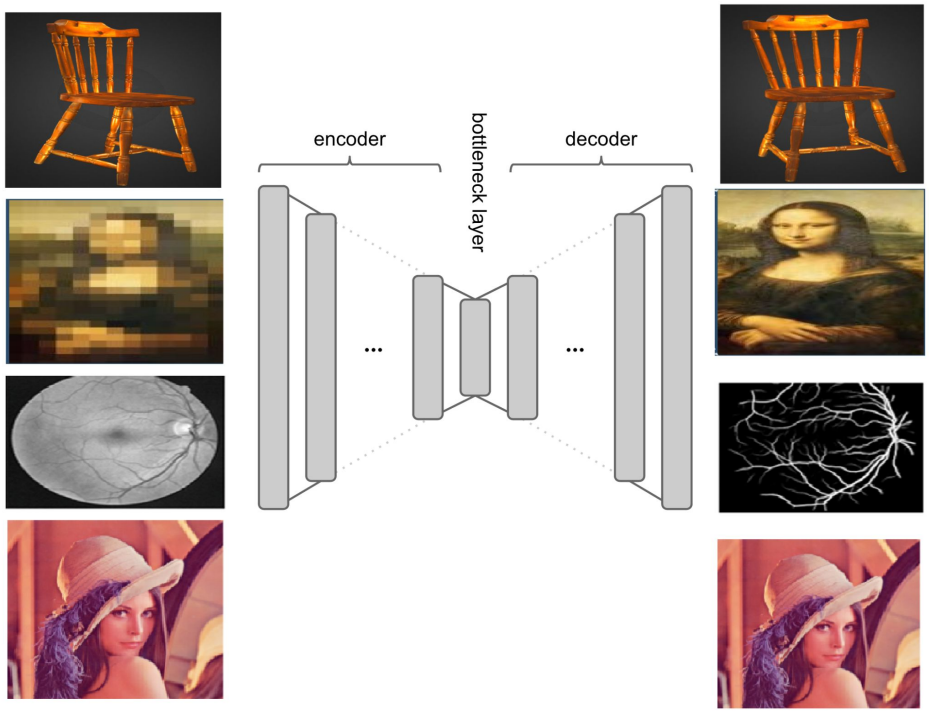
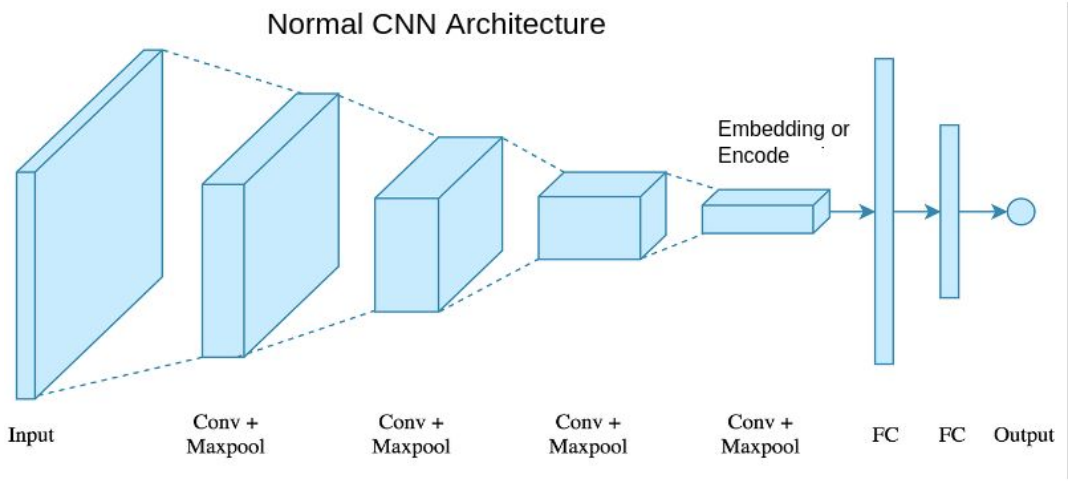


Image Source: Google Search Engine

Embedding / Encode

Encode or embedding or latent space representations can be used for training a smaller network when we have small amount of data or unsupervised training.



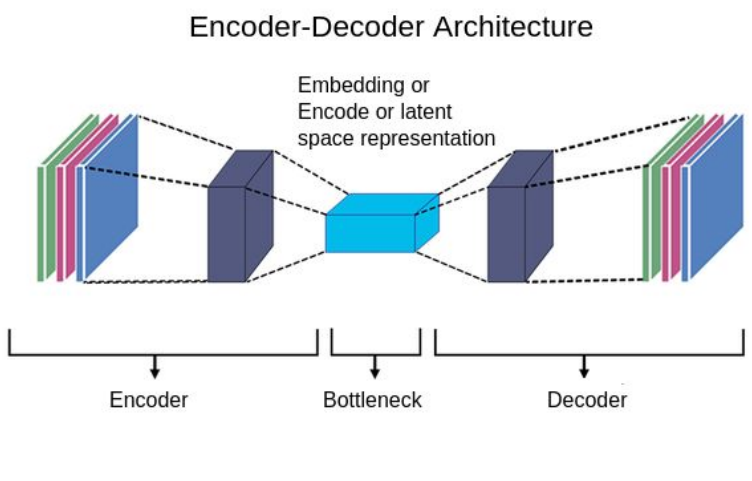
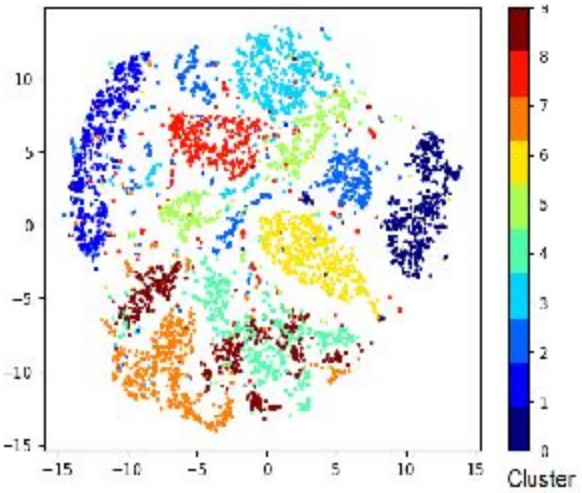


Image Source: Google Search Engine

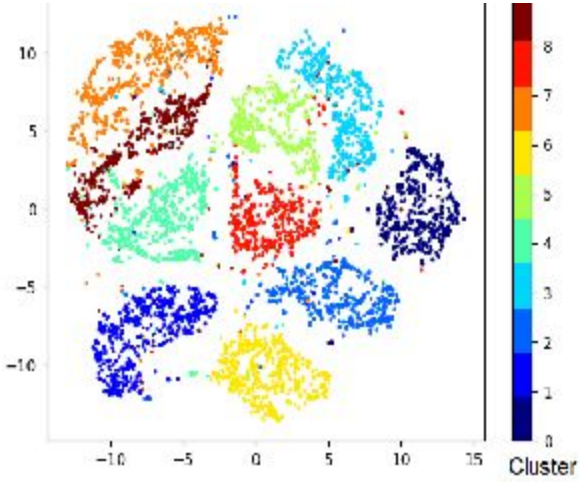
Embedding for Clustering



MNIST Digit Data



k-Means



Autoencoder + k-Means

Image Source: Google Search Engine

Siamese Network

Siamese network is a single network which accepts two or multiple inputs and compares.

It uses same weights for multiple input branches

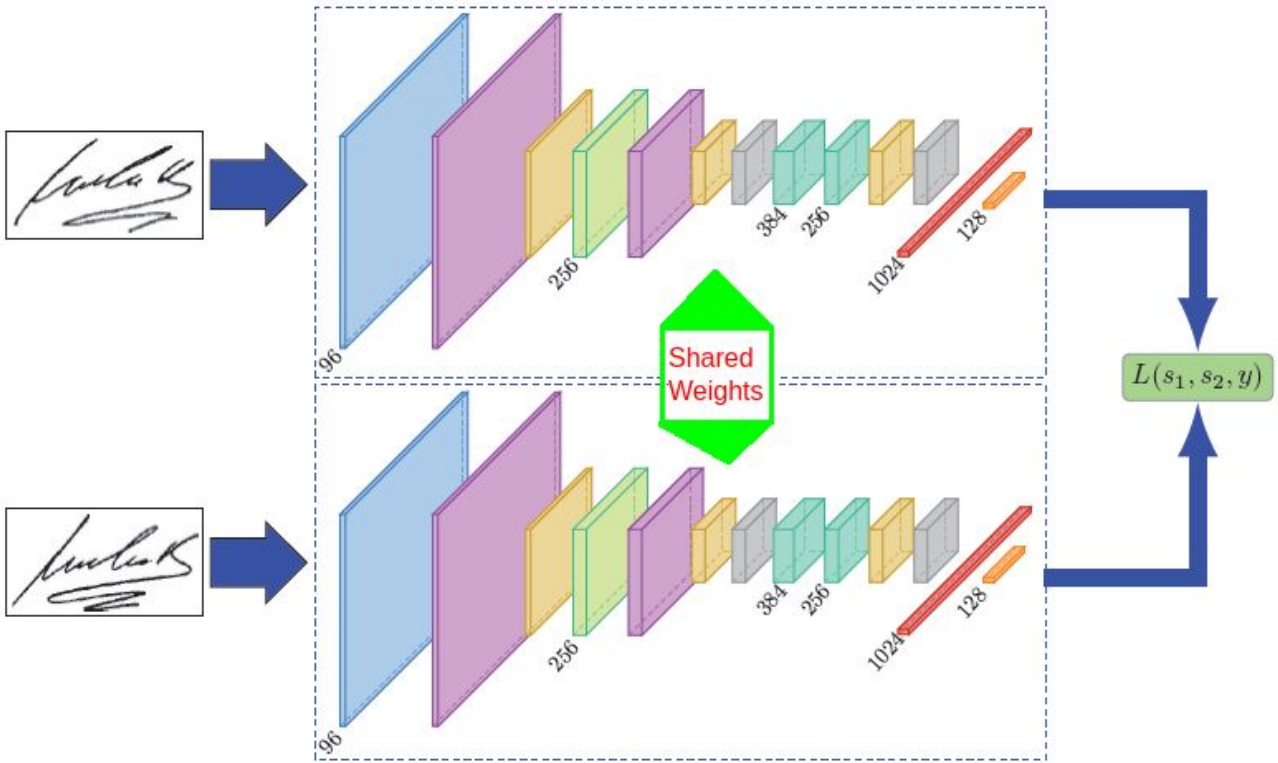


Image Source: Google Search Engine

Self-Supervised vs Supervised Contrastive Loss

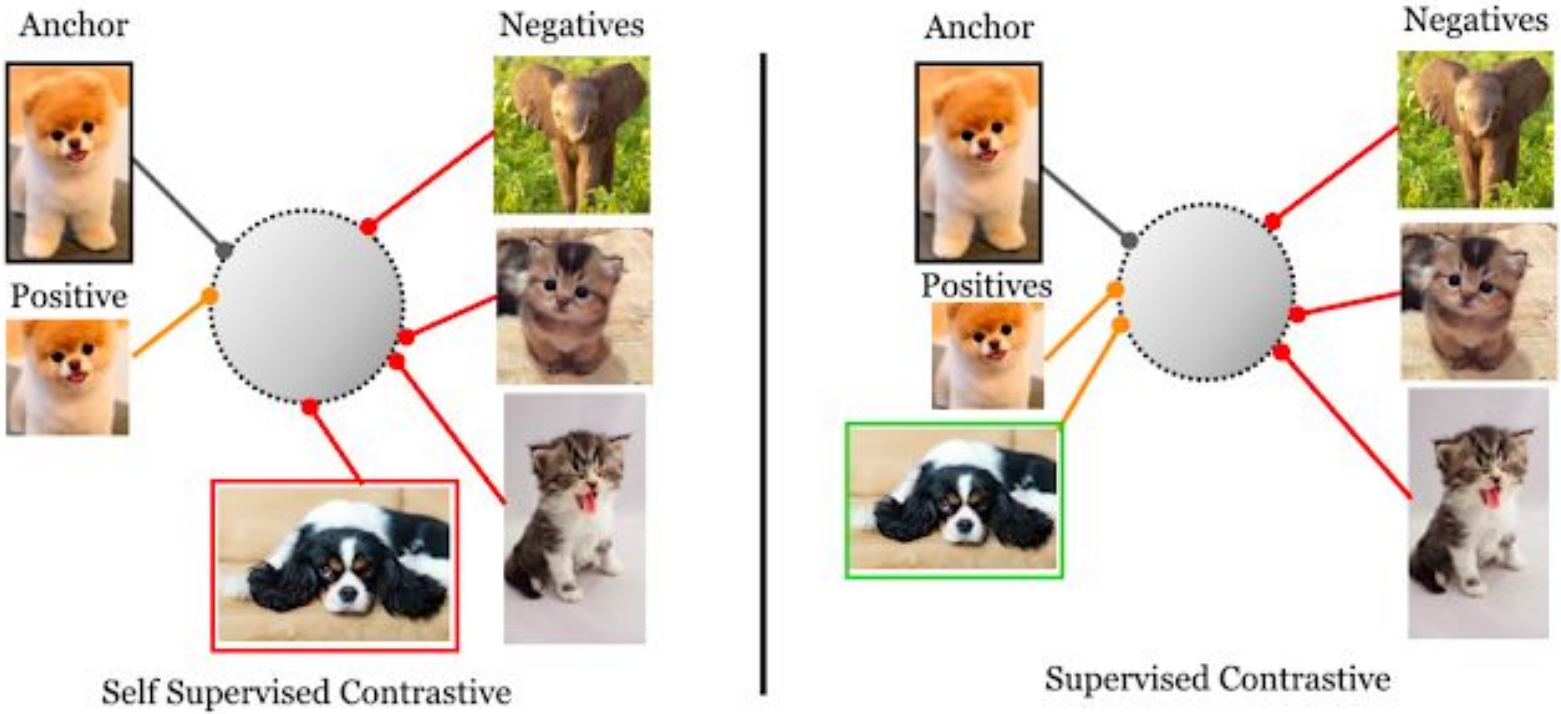


Image Source: Google Search Engine

Triplet Loss

Siamese network uses same weights for multiple input branches

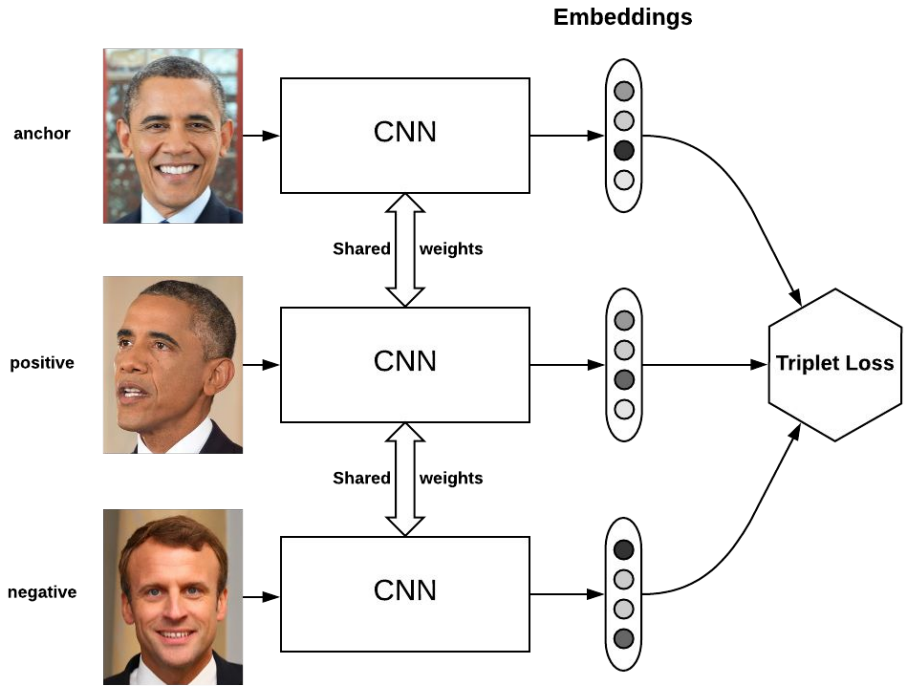
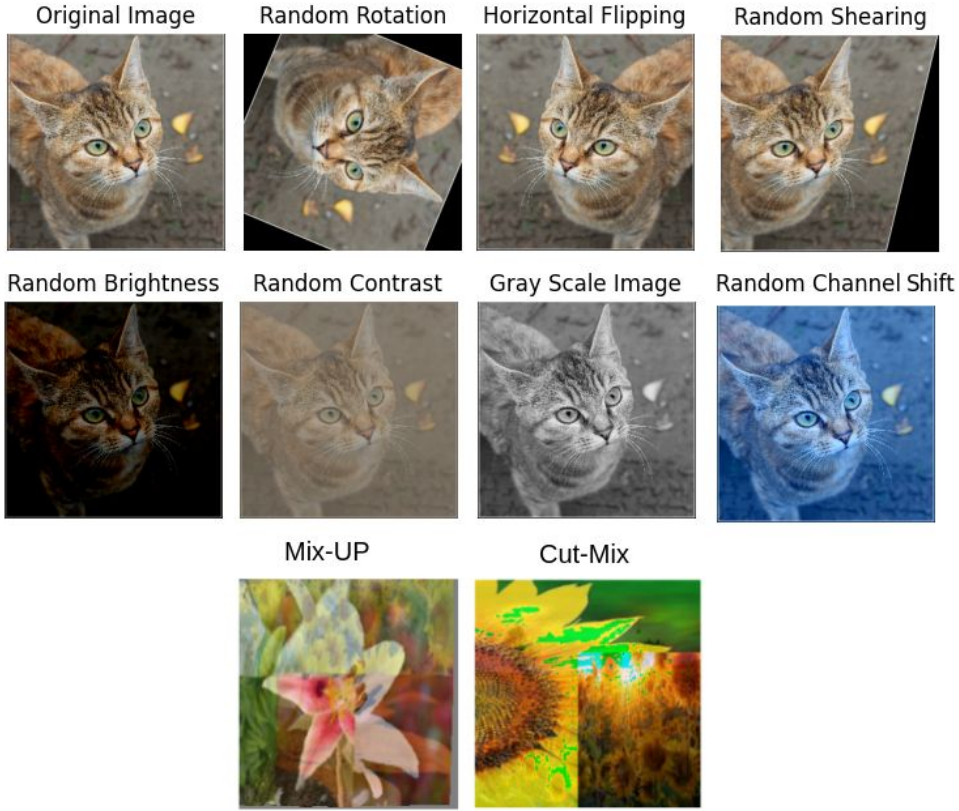


Image Source: Google Search Engine

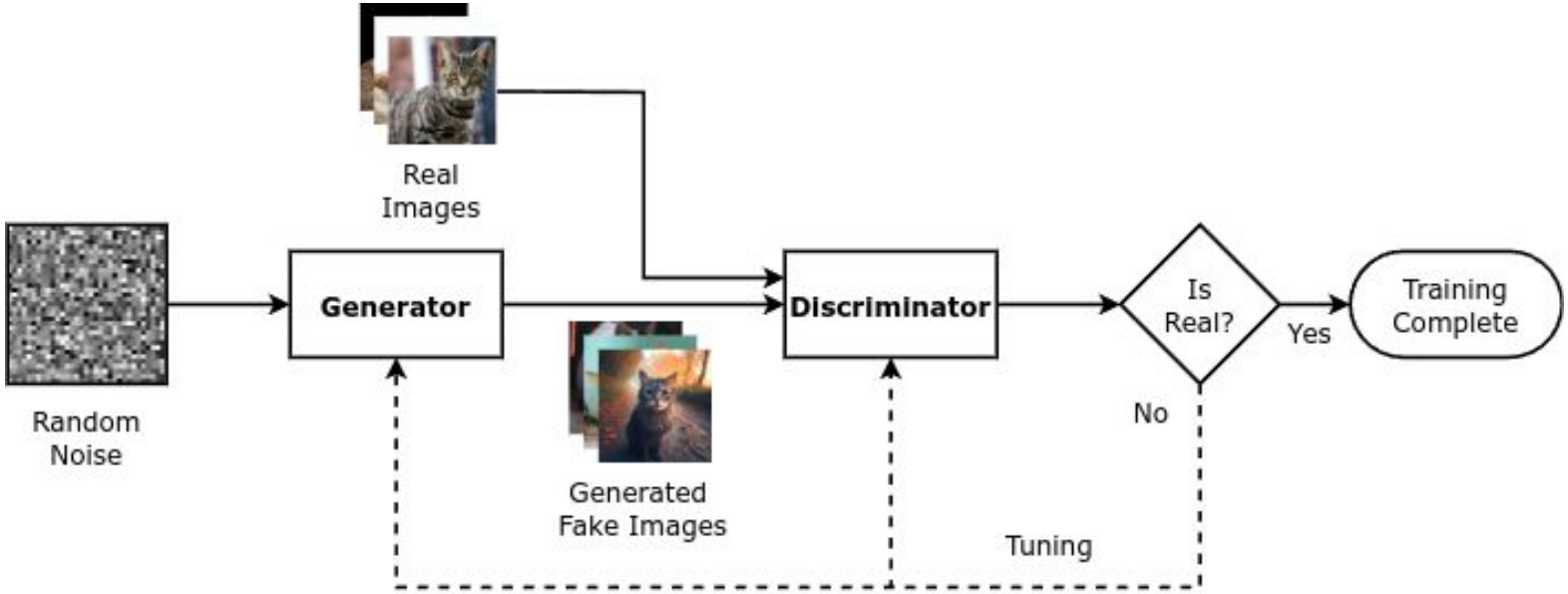
Data Augmentation

- DA is used to increase data
- Random DA should not be used for validation and test sets during validation and testing



Generative Adversarial Neural Network

- Two neural networks are trained one-by-one to beat each other



Explainable NN

Explainable NN help us to understand output of an NN.

Popular methods are:

- Grad-Cam
- Integrated Gradients



Thank You!

(Q&A)